

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Filip Trplan
**Formalizacija originalne formulacije
Poloniusa**

DIPLOMSKO DELO

INTERDISCIPLINARNI UNIVERZITETNI
ŠTUDIJSKI PROGRAM PRVE STOPNJE
RAČUNALNIŠTVO IN MATEMATIKA

MENTOR: doc. dr. Boštjan Slivnik

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil Typst.

Kandidat: Filip Trplan

Naslov: Formalizacija originalne formulacije Poloniusa

Vrsta naloge: Diplomaska naloga na univerzitetnem programu prve stopnje
Računalništvo in matematika

Mentor: doc. dr. Boštjan Slivnik

Opis:

Preverjevalnik izposoj je eden od ključnih delov prevajalnika za Rust, s katerimi se preverja pomnilniško varnost prevajanih programov. Polonius, najnovejši preverjevalnik izposoj, ni nikjer tehnično opisan. Edina točna specifikacija je kar implementacija v trenutnem prevajalniku. Preučite dostopno literaturo in predvsem tudi izvorno kodo Poloniusa v prevajalniku za Rust ter kar se da natančno formalno opišite delovanje Poloniusa.

Title: Formalization of the Original Formulation of Polonius

Description: The borrow checker is one of the key components of the Rust compiler used to ensure the memory safety of compiled programs. Polonius, the newest borrow checker, is not technically described anywhere. Its only precise specification is its implementation in the current compiler. Study the available literature and, in particular, the source code of Polonius in the Rust compiler, and provide as precise a formal description of Polonius as possible.

Zahvala

Zahvaljujem se svojemu mentorju doc. dr. Boštjanu Slivniku za vodstvo, pomoč in temeljite ter koristne popravke ob izdelavi diplomskega dela. Zahvala gre tudi mojim staršem za spodbudo in puncu Klari za vso podporo ter optimizem ob pisanju.

Kazalo

1 Uvod	1
1.1 Motivacijski primer	3
2 Pregled literature	6
2.1 Poskusi formalizacije Rusta	6
2.2 Modeli sorodni lastništvu	8
2.3 Polonius v akademskem svetu in v praksi	9
3 Rustov model upravljanja s pomnilnikom – lastništvo	11
4 Formalizacija Poloniusa	17
4.1 Intuitivna razlaga Poloniusa	17
4.2 Formalizacija pravil	21
4.3 Primer in formalizacija	28
4.4 Osnovne množice in elementi	28
4.4.1 Množica posoj posoje	28
4.4.2 Množica regij regije	30
4.4.3 Graf poteka in množica stavkov stavki	30
4.4.3.1 Primer grafa	31
4.5 Začetne relacije	34
4.5.1 Začetna relacija vsebovanosti	34
4.5.2 Začetna relacija posoje regij	36
4.5.3 Relacija aktivnosti regije	36
4.5.4 Relacija prekinitve posoje	37

4.5.5	Relacija razveljavitve posoje	38
4.6	Izpeljane relacije	38
4.6.1	Relacija vsebovanosti	39
4.6.2	Relacija zahteve	41
4.6.3	Relacija aktivnosti posoje	41
4.6.4	Vizualizacija na primeru	42
4.7	Javljanje napake	43
4.8	Vizualna reprezentacija delovanja Poloniusa	46
5	Zaključek	48
	Literatura	50

Seznam uporabljenih simbolov in kratic

NLL	Non-Lexical Lifetimes (neleksikalne življenjske dobe)
MIR	Mid-level Intermediate Representation (srednjenivojska vmesna predstavitev)
CFG	Control Flow Graph (graf poteka programa)
RFC	Request for Comments (dokument s specifikacijami)

Povzetek

Naslov: Formalizacija originalne formulacije Poloniusa

Avtor: Filip Trplan

Povzetek:

Naloga zastavi matematično formalizacijo Poloniusa, preverjevalnika izposoj za programski jezik Rust. Posebnost Rusta je njegov sistem tipov, ki prevajalniku z ustreznimi pravili o izposojevanju omogoča zagotavljanje pomnilniške varnosti že v času prevajanja. Trenutna implementacija preverjevalnika izposoj, imenovana NLL, je v nekaterih primerih preveč konzervativna, zato so razvijalci Rusta uvedli novo različico, imenovano Polonius, ki je osnovana na bolj natančni analizi toka podatkov. Polonius sicer nikjer ni uradno definiran, viri o njem so razpršeni, zato je cilj te naloge postaviti matematičen okvir, skozi katerega lahko razumemo to novo različico. Tega se lotimo z uporabo množic in izjav, tako da pravila, ki so bila zastavljena v raznih virih, opišemo s pomočjo predikatov ter pravil sklepanja. Končni izdelek je poenostavljen, vendar formalen opis Poloniusa.

Ključne besede: Rust, Polonius, preverjevalnik izposoj, formalizacija

Abstract

Title: Formalization of the Original Formulation of Polonius

Author: Filip Trplan

Abstract:

This thesis provides a mathematical formalization of Polonius, the next-generation borrow checker for the Rust programming language. Rust's distinguishing feature is its ability to guarantee memory safety through its type system and borrow checker, ensuring safety without impacting runtime performance. The current implementation, known as NLL (Non-Lexical Lifetimes), remains overly conservative in certain cases – consequently, Rust developers have introduced a new version called Polonius, which is based on a more precise data-flow analysis. However, Polonius lacks an official specification, and information regarding its workings remains scattered. The goal of this thesis is to establish a mathematical framework that enables a clear understanding of this new implementation. This is approached using sets and logical statements, describing the rules established in various sources through predicates and inference rules. The final result is a simplified but formal description of Polonius.

Keywords: Rust, Polonius, borrow checker, formalization

Poglavje 1

Uvod

Pomnilniška varnost (angl. memory safety) je na področju razvoja programske opreme vedno aktualna tema. Razvijalci pri Microsoftu so pokazali, da so napake pri upravljanju s pomnilnikom najpogostejši tip napak [1]. Pri projektu Chromium, na katerem temelji Google Chrome, so ugotovili, da približno 70 odstotkov hroščev povzročijo tovrstne napake [2]. Če bi se lahko znebili te kategorije napak, bi se izognili precejšnjemu delu hroščev. Posledično so se razvijalci programskih jezikov pomnilniške varnosti lotili na različne načine.

Eden najbolj razširjenih jezikov je C, kjer je upravljanje pomnilnika povsem prepuščeno programerju. Tovrsten pristop, imenovan ročno upravljanje pomnilnika, lahko vodi do izredno hitrih programov in krajših časov prevajanja v primerjavi z Rustom [3], vendar je obenem tudi pogost vir napak [2].

Alternativni pristop ročnemu upravljanju je avtomatsko upravljanje pomnilnika, kjer programski jezik zagotavlja varno dodeljevanje in sproščanje pomnilnika. S tem razbremeni programerja, ki se lahko osredotoči na pisanje programa. Vendar imajo jeziki z avtomatskim upravljanjem pomnilnika dve glavni slabosti: zaradi zakasnjene sproščanja se pojavi večja poraba pomnilnika, obenem pa prihaja do premorov med izvajanjem programa ali

zakasnitev ob vsaki operaciji, da lahko čistilec pomnilnika najde pomnilniške lokacije za sprostitev pomnilniških blokov [4].

Rust pristopa k upravljanju s pomnilnikom na drugačen način. Veljavnost dostopanja do pomnilniških lokacij se preverja med prevajanjem s pomočjo preverjevalnika izposoj (angl. borrow checker). To je komponenta Rustovega prevajalnika, ki se ukvarja s tokom podatkov in pomnilniškimi lokacijami. Rust imenuje zbirko pravil, ki opisuje delovanje preverjevalnika izposoj, lastništvo (angl. ownership). V knjigi *The Rust Programming Language* avtorji lastništvo opišejo tako: „*Ownership is a set of rules that govern how a Rust program manages memory*“ [5]. Tak pristop ima dve glavni prednosti: zagotavlja, da je program pomnilniško varen kot pri avtomatskem upravljanju s pomnilnikom, ter omogoča hitrost izvajanja programov, ki jo lahko dosežemo z ročnim upravljanjem pomnilnika [5]. Pogosto omenjena slabost Rusta je dolg čas prevajanja [3]. Ta sicer ni odvisen samo od preverjevalnika izposoj, vendar njegov prispevek ni zanemarljiv.

V nadaljevanju bomo uporabljali dva podobna pojma. *Varen program* je program, ki ne povzroča pomnilniških napak. *Veljaven program* pa je program, ki ustreza Rustovim pravilom lastništva in izposojanja. Cilj Rustovega prevajalnika je, da bi bili ti dve množici programov enaki. Ob predpostavki, da so Rustova pravila lastništva in izposojanja pravilna, je vsak veljaven program v Rustu tudi varen, zaradi neizračunljivosti pa žal vsak varen program v Rustu ni veljaven.

Preverjevalnik izposoj, ki je glede na razvoj Rusta hkrati definicija in implementacija pravil lastništva in izposojanja, se je med razvojem Rusta bistveno spremenil od svoje prvotne implementacije. Na začetku je bil preprost in zaradi svoje konzervativnosti pri zagotavljanju varnosti veliko varnih programov zavrnil [6]. Zato se je čez nekaj let pojavila naslednja različica preverjevalnika, imenovana NLL (angl. non-lexical lifetimes), ki je rešila veliko pogostih problemov prvotne različice. Vendar NLL še vedno ni sprejemal vseh varnih programov. Da bi to izboljšali, so Rustovi razvijalci predlagali trenutno najnovejšo različico preverjevalnika, imenovano

Polonius, ki drugače zastavi problem lastništva in tako sprejme še večji delež varnih programov [7].

NLL je bil natančno opisan v RFC-ju (angl. request for comment), kar je potem vodilo njegov razvoj, Polonius pa je nastal kot predlog na spletnem blogu enega izmed razvijalcev Rusta, kjer se je postopoma razvijal skozi nadaljnje objave [8, 9, 10]. Celovit centraliziran formalen opis Poloniusa trenutno ne obstaja, imamo le nekaj spletnih objav, delni formalni opis v magistrskem delu enega izmed razvijalcev [11], nedokončano knjigo na GitHubu [10] in trenutno implementacijo v Rustovem prevajalniku.

Cilj te naloge je torej na svoj način formalizirati pravila, na katerih temelji Polonius. Najprej raziščemo pretekle poskuse formalizacije Rusta ter sorodne načine upravljanja pomnilnika. Sledi intuitivni opis Rustovih pravil izposojanja in nato formalni opis Poloniusovih inferenčnih pravil.

V preostanku naloge od bralca pričakujemo osnovno znanje programskega jezika Rust. Ker Rust nima uradne specifikacije jezika, se bomo tudi v prihodnje dostikrat sklicevali na izvirno kodo prevajalnika, ki trenutno služi kot edini vir, ki določa delovanje Rusta (angl. specification as implementation). V tej nalogi uporabljamo različico prevajalnika 1.94.0.

1.1 Motivacijski primer

Za grajenje intuicije o razlikah med trenutno različico preverjevalnika izposoj (NLL) in Poloniusom bomo obravnavali motivacijski primer (program 1), ki ga NLL zavrne, Polonius pa sprejme. Primer je prilagojen iz prvotnega predloga NLL-ja [6].

Če postopoma sledimo sporočilu o napaki na izpisu 2, lahko vidimo, kje in zakaj NLL ne sprejme varnega programa. V vrstici 8 kličemo funkcijo `get_mut`, ki vrne unijo z dvema možnostima. Lahko vrne unikatno referenco na vrednost, ki pripada ključu (`Some(value)`), ali pa ne vrne ničesar (`None`). Če vrne vrednost, je spremenljivka `map` začasno izposojena (torej obstaja unikatna referenca na pomnilniško lokacijo z njenimi podatki), kar se zgodi

```

1  fn process(val: &mut String) {
2      unimplemented!();
3  }
4
5  fn process_or_default<'a>(map: &'a mut HashMap<&str, String>)
6      -> &'a mut String {
7      let key = "test";
8      match map.get_mut(&key) { // -----+ 'lifetime
9          Some(value) => value,           // |
10         None => {                       // |
11             map.insert(key, String::default()); // |
12             // ^ ERROR.                // |
13             map.get_mut(&key).unwrap() // |
14         }                               // |
15     } // <-----+
16 }

```

Program 1: Motivacijski primer za Polonius

v vrstici 9. Vendar NLL presodi, da je spremenljivka `map` še vedno izposojena, tudi če nismo vrnili njene reference iz funkcije `get_mut` (v vrsticah 11-13). Ko torej poskušamo vstaviti nov par v `map`, nam to preverjevalnik izposoj konzervativno prepreči, saj operacija `insert` zahteva unikatno referenco na spremenljivko `map` (ker jo spreminjamo z vstavljanjem para), dve unikatni referenci na isto mesto pa po pravilih jezika ne smeta obstajati.

V nasprotju z NLL-jem Polonius prevede program 1 kot veljaven, saj ima večje zmožnosti sledenja kontrolnemu toku in lahko zgornjo analizo opravi podrobneje. NLL ima omejene zmožnosti obravnavanja kontrolnega toka, ki jih Polonius nadgradi v zameno za hitrost preverjanja izposoj v času prevajanja. Amanda Stjerna, ena izmed razvijalcev Poloniusa, je na predstavitvi na konferenci EuroRust omenila, da v prihodnosti načrtujejo dvoslojni preverjevalnik izposoj. Med prevajanjem bi se najprej analiza opravila z NLL-jem, saj je bistveno hitrejši, Polonius pa bi potem obravnaval samo zahtevnejše primere, ki bi jih NLL zavrnil [12] (na 23:15).

```
1 error[E0499]: cannot borrow `*map` as mutable more than
  once at a time
2 --> /tmp/IWXsFebCZD/main.rs:11:13
3 |
4 5 | fn process_or_default<'a>(map: &'a mut HashMap<&str,
  String>)
5 | |                                     -- lifetime ``a` defined here
6 | ...
7 8 |     match map.get_mut(&key) { // -----+ 'b
8 |     -     --- first mutable borrow occurs here
9 |     | _____|
10 |     | |
11 9 | |         Some(value) => value,           // |
12 10 | |         None => {                       // |
13 11 | |             map.insert(key, String::default()); // |
14 | |             ^^^ second mutable borrow occurs here
15 | ... |
16 14 | |         }                               // |
17 15 | |     } // <-----+
18 | | _____- returning this value requires that `*map` is borrowed
  for ``a`
```

Izpis 2: Obvestilo o napaki pri prevajanju programa 1 z NLL-jem

Poglavje 2

Pregled literature

Rust je jezik inženirjev, ne raziskovalcev. Od začetka je bil zasnovan tako, da reši današnje probleme ter se ukvarja s specifikacijami in formalnostjo kasneje. Ta način dela je porodil veliko vprašanj o tem, kako jezik deluje, zakaj deluje in ali sploh deluje pravilno. Čeprav je Rust prišel na svet šele leta 2015 [13], je v zadnjem desetletju nastalo vrsto člankov o raznih formalnih pogledih na Rust.

V tem poglavju se bomo lotili treh glavnih kategorij raziskav in virov:

1. **Poskusi formalizacije Rusta:** Ogledali si bomo, kako so se raziskovalci lotili problema formalizacije različnih komponent Rusta.
2. **Modeli sorodni lastništvu:** Ker je Rustov pomnilniški model eden izmed njegovih ključnih značilnosti, si bomo ogledali sorodne modele.
3. **Polonius v akademskem svetu in praksi:** Tukaj bomo opisali, kako je Polonius nastal, kje trenutno je in drugo literaturo o njem.

2.1 Poskusi formalizacije Rusta

Formalizacija Rusta in preverjanje pravilnosti kode sta zelo relevantni temi, saj je jezik kompleksen in brez uradne specifikacije. Osredotočili se bomo na formalizacije preverjevalnika izposoj, saj je to tema te naloge. Obstajajo tudi

številne druge formalizacije, ki se ukvarjajo s sistemom tipov ali pravilnostjo programov [14, 15].

V nadaljevanju bomo omenili vmesno kodo *MIR* (angl. Mid-level intermediate representation), s pomočjo katere prevajalnik preverja pomnilniško pravilnost programov. *MIR* je bistveno poenostavljena oblika Rusta in zadnji korak pred generiranjem strojne kode v zadnjem delu prevajalnika (v Rustovem primeru je zadnji del LLVM). Temelji na grafu kontrole toka, ki ga opišemo pozneje v nalogi.

Še en pomemben pojem je *zataknjeno stanje* (angl. stuck state), ki intuitivno pomeni, da program ne more nadaljevati, saj iz trenutnega stanja glede na operacijsko semantiko jezika ni več veljavnega koraka. Torej je stanje glede na definicijo jezika nesmiselno [16].

Eno izmed ključnih del na področju formalizacije je članek *RustBelt: securing the foundations of the Rust programming language*, v katerem so avtorji zasnovali jezik imenovan λ_{Rust} ter ga opremili s semantičnim modelom imenovanim RustBelt. Jezik λ_{Rust} je sam bolj podoben MIRu kot pa izvirni kodi Rusta. Vsebuje tudi sistem tipov in pravila sklepanja, ki modelirajo *MIR*. Članek se konča z dokazom, da katerikoli λ_{Rust} program, ki je semantično in tipsko pravilen, ne bo končal v zataknenem stanju [17].

Še en model Rusta je imenovan Oxide [18], kjer avtorji zasnujejo višjenivojski jezik, tokrat bolj podoben izvirni kodi Rusta. V primerjavi z RustBeltom se avtorji bolj osredotočijo na preverjevalnik izposoj, saj niso želeli natančno modelirati operacijske semantike, temveč je bil njihov cilj zajeti bistvo Rusta. Oxidova sintaksa je zelo podobna Rustovi, le da so vsi tipi eksplicitno podani. Avtorji nadaljujejo članek s tem, da podajo pravila sklepanja v tem sistemu tipov in uvedejo pojem *domnevnega izvora* (angl. approximate provenance), ki je njihov način izražanja regij, kot so zastavljene v NLL-ju. Članek se nadaljuje s semantiko majhnih korakov in konča s formalnim dokazom, da pravilno konstruirani programi v Oxidu ne končajo v zataknenem stanju. Pri tem članku je še zanimivo, da specifično omenijo Polonius ter povedo, da je Poloniusov model regij zelo podoben

njihovim domnevnim izvorom. Omenijo, da kljub temu, da niso raziskali povezave med Poloniusom in OxiDOM, lahko na Oxide gledamo kot na formulacijo Poloniusa preko sistema tipov.

Takih podobnih modelov je še mnogo. Članek Crichtona idr. zastavi poenostavljen pedagoški model za razumevanje sistema lastništva [19]. V njem tudi eksplicitno, vendar le na kratko, opišejo Polonius. Patina je eden izmed prvih formalnih semantičnih modelov za Rust in je nastala še pred uvedbo NLL-ja [20]. KRust [21] je formalni izvedljiv semantični model v ogrožju imenovanem K in je, kolikor vemo, prvi formalni semantični model NLL-ja. Featherweight Rust [22] se eksplicitno ukvarja s formalizacijo preverjevalnika izposoj (vrste NLL), tako da definira slovnico in semantiko majhnih korakov. Zasnovan je bil kot striktna podmnožica Rustove sintakse. Še zadnji članek, ki ga bomo omenili, vsebuje dokaz, da LLBC (low-level borrow calculus, model Rustovega MIR) res pravilno modelira Rust in ne konča v zataknjenem stanju [23].

2.2 Modeli sorodni lastništvu

V tem razdelku se bomo osredotočili na *regijsko upravljanje pomnilnika* (angl. region-based memory management) [24], ki sta ga prva opisala Tofte in Talpin. Ta model upravljanja s pomnilnikom lahko razumemo skoraj kot neposredni predhodnik lastništva, kot se uporablja v Rustu.

Njuna poglobljena motivacija je bila, da najdeta kompromis med ročnim upravljanjem s pomnilnikom, kot je to pri C-ju, ter avtomatskim čiščenjem pomnilnika, kot je to pri Javi. Za navdih sta vzela delovanje sklada, kjer se klicni zapis dodeli na začetku izvajanja funkcije ter sprosti na koncu. Tako sta ustvarila koncept regij, ki so dodatne označbe poleg tipov in podajo informacije o tem, kdaj se mora vrednost sprostiti.

Ker bi bilo anotiranje vsake vrednosti z regijami nepraktično, sta uvedla način avtomatskega izračuna regij, podoben tistemu, ki izračuna življenjske dobe v Rustu. V delu definirata visokonivojski jezik `SExp`, podoben SML-u,

skupaj s sistemom ML tipov in semantiko majhnih korakov. Nato uvedeta jezik TExp, v katerega se SExp pretvori. Ključna razlika med njima je, da ima TExp regijske anotacije, SExp pa ne. Delo nadaljujeta s sistemom za avtomatično izpeljevanje teh regij, osnovanem na Milnerjevem sistemu tipov. Zaključita z dokazi o pravilnosti njunega sistema in pravilnosti prevoda med SExp in TExp.

Rust ni bil prvi jezik, ki je uvedel pomnilniški model soroden regijskemu upravljanju s pomnilnikom (poleg seveda akademskega jezika, predstavljenega v izvornem delu). Eden izmed najbolj znanih jezikov, ki so v praksi uporabili regijsko upravljanje pomnilnika, je Cyclone [25]. Ustvarjen je bil kot dopolnilo C-ju z raznimi naprednimi tipi. Kasneje so dodali regijsko upravljanje pomnilnika, ki ga lahko programer doda C programu z nekaj dodatnimi regijskimi anotacijami. Prva implementacija sicer ni bila popolna in je še vedno včasih povzročila puščanje pomnilnika (angl. memory leaks), zato so kasnejše različice jezika z linearnimi regijami to poskušale popraviti [26].

2.3 Polonius v akademskem svetu in v praksi

Polonius je bil prvotno formuliran v spletni objavi N. D. Matsakisa, kjer je ta poljudno pojasnil, kako bi Polonius naslovil problem starega NLL, in podal osnovno formulacijo v Datalogu [7]. Delo se je nato nadaljevalo v GitHub repozitoriju `rust-lang/polonius` [27], kjer so to originalno formulacijo implementirali v Rustu.

Leta 2020 je Amanda Stjerna v svojem magistrskem delu podala prvo matematično formulacijo Poloniusa kot sistema tipov [11]. Ta formulacija je bila močno osnovana na Oxidu, saj sta si modela zelo podobna. V svojem delu je opisala tudi pravila za preverjevalnik izposoj, ki jih kasneje v nalogi opišemo in formaliziramo. Njeno delo se nadaljuje z natančnejšim opisom Poloniusovega notranjega delovanja z vsemi podrobnostmi, potrebnimi za

konkretno implementacijo. Kolikor vemo, je to delo eno izmed najbolj podrobnih in celovitih opisov Poloniusovega delovanja.

Trenutno še ne obstaja uradna specifikacija za Polonius, vendar se stanje počasi spreminja. Leta 2025 je bil Polonius implementiran v glavni veji Rustovega prevajalnika in je trenutno na zahtevo dostopen v nočni (angl. *nightly*) različici [28]. V izvorni kodi prevajalnika sta prisotni dve različici Poloniusa [29]: osnovna (angl. *legacy*) različica, podobna prvotni implementaciji iz [27], ter razvojna (angl. *alpha*) različica, ki je nastala kot odgovor na počasno izvajanje prve. Slednja je napisana neposredno v Rustu in ne emulira Dataloga, a zato tudi ne podpira vseh zmogljivosti, ki jih ponuja osnovna različica [28].

Pomanjkanje uradne specifikacije je problem, ki ga trenutno rešuje Rustova ekipa za tipe v okviru projekta, imenovanega *a-mir-formality* [30, 31]. Želijo ustvariti uradno izvedljivo specifikacijo za Rust, s katero se bo potem preverjalo pravilno delovanje Rustovega prevajalnika. „Izvedljiva“ v tem kontekstu pomeni, da ji lahko kot vhod damo Rust program (oziroma trenutno MiniRust, ki je bolj podoben MIRu [32]), specifikacija pa ga nato sprejme ali zavrne, glede na to, ali je pravilno tipiziran in pomnilniško varen. V drugi polovici leta 2025 se je začelo delo na specifikaciji za razvojno različico Poloniusa, ki je v času pisanja na začetku 2026 že skoraj končana. Če se delo pod *a-mir-formality* nadaljuje, bo lahko Rust končno dobil uradno specifikacijo, ki mu že od spočetja manjka.

Poglavje 3

Rustov model upravljanja s pomnilnikom – lastništvo

V razdelku 1.1 smo s programom 1 predstavili primer, ki je motiviral nastanek Poloniusa. Podali smo tudi intuitivno razlago, zakaj je ta program varen, kjer smo se nanašali na pravila, osnovana na lastništvu – Rustovem naboru pravil za zagotavljanje pomnilniško varnih programov.

Knjiga *The Rust Programming Language*, neuradni priročnik za Rust, pojasnjuje, da lastništvo temelji na treh pravilih [5]:

1. Vsaka vrednost v Rustu ima *lastnika*.
2. Za vsako vrednost lahko obstaja le en lastnik hkrati.
3. Ko lastnik ni več v dosegu, je vrednost sproščena (angl. dropped).

Lastnik se tukaj nanaša na spremenljivko (natančneje lvalue o. levo vrednost), na katero je ta vrednost vezana. V programu 3 opazimo, da vrednost "hello" enkrat zamenja lastnika, torej njen prvotni lastnik `a` potem ne vsebuje več vrednosti, saj je ta zdaj v lasti `b`. Če želimo uporabiti `a` potem, ko ni več lastnik vrednosti, nam prevajalnik javi napako.

Lastništvo je vezano na doseg. Koncept dosega lahko preprosto ponazorimo z leksikalnim dosegom, tako da inicializiramo novo vrednost, vezano

```
1 let a = "hello";  
2 let b = a;  
3 println!("{}", a); // prevajalnik javi napako
```

 Rust

Program 3: Primer napačnega lastništva

```
1 {  
2     let a = "goodbye";  
3 }  
4 println!("{}", a); // prevajalnik javi napako, ker `a` ni več v  
   dosegu
```

 Rust

Program 4: Primer leksikalnega dosega

na `a`, znotraj gnezdenega bloka, ki ustvari nov doseg. To je prikazano v programu 4.

Za razliko od C in C++, ki takšnih napak ne zaznata v času prevajanja, Rust lastništvo uveljavlja že v fazi prevajanja. Dodatna razlika se še pojavi pri ustvarjanju referenc ter njihovi delitvi na dva različna tipa. Rustove reference so na prvi pogled podobne kazalcem, kakršne poznamo iz drugih programskih jezikov. Ključna razlika je v tem, da Rustov prevajalnik zagotovi, da referenca vedno kaže na veljavno vrednost pravega tipa – in to skozi celotno življenjsko dobo te reference [5]. Ta varnostni mehanizem omogoča nekaj, kar je v mnogih drugih jezikih bistveno težje doseči: zagotovilo, da reference „ne visijo v prazno“ in da ne dostopamo do podatkov, ki morda sploh več ne obstajajo.

V preostanku naloge ima MIR osrednjo vlogo, saj bistveno poenostavi preverjanje izposoj in omogoča lažjo analizo. V okviru MIRa je tudi definiran naslednji pojem:

Mesto (angl. *place*) Mesto je izraz, ki opredeli lokacijo v pomnilniku.

To je lahko lokalna spremenljivka (npr. *oseba*) ali pa njena projekcija

(npr. polje strukture `oseba.starost`) [33]. To je eden ključnih pojmov pri analizi pomnilniške varnosti programa.

Zdaj lahko s pojmom mesta opredelimo dve glavni vrsti referenc [18, 19, 34]. Delimo jih lahko na dveh oseh: spremenljive ali nespremenljive in unikatne ali deljene. Ker slednja delitev bolje ponazori omejitve pri ustvarjanju referenc, bomo uporabljali naslednjo terminologijo:

Deljene reference (angl. shared references) To so reference, ki nam omogočajo, da ustvarimo več referenc na isto mesto hkrati. Zato morajo biti tudi *nespremenljive* (angl. immutable), kar pomeni, da podatkov na referenciranem mestu ne smemo spreminjati. To pravilo mora veljati, da je uporaba tovrstnih referenc varna.

Unikatne reference (angl. unique references) To so reference, ki zagotovijo, da obstaja samo ena referenca na mesto hkrati. Občasno želimo tudi spreminjati vrednost, na katero kaže referenca preko te reference. Zato uvedemo unikatne reference, ki so posledično *spremenljive* (angl. mutable). Pravilo, ki ohranja pomnilniško varnost, se glasi: če obstaja unikatna referenca na pomnilniško mesto, na to mesto ne sme kazati nobena druga aktivna referenca (deljena ali unikatna). Aktivnost reference tukaj pomeni isto kot aktivnost spremenljivke.

Opomba: V rustovski terminologiji se reference običajno ne ločijo po isti osi. Navadno jih delimo na deljene ter nespremenljive reference.

Programi 5-8 ponazarjajo pravilno in nepravilno uporabo različnih tipov referenc. Če najprej obravnavamo programa 5 in 6, vidimo, kako se pravilo o nespremenljivosti izrazi v kodi. Če bi v programu 5 po izpisu dodali še vrstico `*b = 7` (pisanje prek reference), bi nam prevajalnik vrnil napako zaradi kršitve zagotovila o preprečitvi pisanja.

V programih 7 in 8 prav tako opazimo, da če bi poskusili izpisati spremenljivko `a`, ki je bila unikatno izposojena v programu 7, bi bil program zavržen, saj prevajalnik prepreči, da bi hkrati uporabljali lastnika vrednosti in njeno unikatno referenco.

```
1 let a = 6;
2 let b = &a; // ustvarimo deljeno referenco
3 println!("{}", a); // lahko jo uporabimo, ker za izpis na
4               // zaslon potrebujemo samo branje
```

 Rust

Program 5: Pravilna uporaba deljene reference

```
1 let a = 6;
2 let b = &a; // ustvarimo deljeno referenco
3 *b = 7; // NAPAKA: poskus pisanja skozi deljeno referenco
4 println!("{}", a);
```

 Rust

Program 6: Napačna uporaba deljene reference – poskus pisanja

```
1 let mut a = 6;
2 let b = &mut a;
3 *b = 7; // lahko spremenimo podatke na pomnilniški lokaciji,
4         // ker je referenca unikatna
```

 Rust

Program 7: Pravilna uporaba unikatne reference

```
1 let mut a = 6;
2 let b = &mut a; // ustvarimo unikatno referenco
3 println!("{}", a); // NAPAKA: hkratna uporaba lastnika in unikatne
4   reference
4 *b = 7;
```

 Rust

Program 8: Napačna uporaba unikatne reference – hkratna uporaba lastnika

To razmerje med obema vrstama referenc – večkratne nespremenljive ali ena sama spremenljiva – lahko strnemo v načelo, ki ga v angleščini imenujemo *aliasing XOR mutability*. Ideja tega načela je preprosta: podatkovne strukture so lahko bodisi dostopne na več mestih hkrati (torej imajo več

```

1  fn main() {
2      let r;           // -----+-- 'a
3                      //      |
4      {               //      |
5          let x = 5;   // -+-- 'b |
6          r = &x;      // |      |
7      }              // -+      |
8                      //      |
9      println!("r: {r}"); //      |
10 }                  // -----+

```

Program 9: Pripisane življenjske dobe

imen oziroma referenc), vendar jih lahko samo beremo, bodisi jih smemo aktivno spreminjati, vendar z zagotovitvijo, da ima v tistem trenutku do njih dostop le ena referenca. Model tako na zelo eleganten način povezuje podatke z naborom dovoljenih operacij in to počne prek samega sistema tipov [34].

Življenjske dobe (angl. lifetimes) Še ena podrobnost, ki je pomembna za razumevanje lastništva, so življenjske dobe, ki so v Rustu sestavni del tipov. Kot sami tipi v Rustu so ponavadi izpeljane, vendar se pogosto pri podpisu funkcije zgodi, da jih moramo eksplicitno pripisati. Dejanski tip reference na niz ni `&String`, ampak `&'a String`, kjer je `'a` življenjska doba. Življenjske dobe so sicer del tipa samo takrat, ko ta predstavlja referenco. Intuitivno si jih lahko predstavljamo kot nabor vrstic v programu, kjer mora biti ta referenca veljavna [5]. Koncept življenjskih dob kot nabor vrstic predstavimo s programom 9.

Prevajalnik nam pri programu 9 vrne napako, saj je spremenljivka `x` veljavna samo za življenjsko dobo `'b`, vendar prevajalnik ugotovi, da mora biti veljavna za `'a`, saj se uporabi pri izpisu na zaslon. V gnezdenem bloku dodelimo tipu `&'a i32` vrednost tipa `&'b i32`, vendar slednja ni podtip prve, saj je nabor vrstic `'b` stroga podmnožica nabora `'a`. Izračun življenjskih dob je odvisen od implementacije preverjevalnika izposoj, vendar si jih

lahko intuitivno predstavljamo kot najmanjšo množico vrstic, kjer bo ta spremenljivka oziroma mesto še uporabljeno.

Poglavje 4

Formalizacija Poloniusa

V tem poglavju najprej predstavimo intuitivni opis delovanja Poloniusa, temu pa sledi formalni opis pravil Rustovega preverjevalnika izposoj. Nazadnje predstavimo formalizacijo Poloniusa z opisom na osnovi množic in relacij.

4.1 Intuitivna razlaga Poloniusa

Preden formalno opišemo vse podrobnosti Poloniusa, je pomembno pridobiti nekaj intuicije o njegovem delovanju, saj nam bo to olajšalo razumevanje pravil, na katerih algoritem temelji. Naslednjo razlago smo povzeli iz spletne objave, ki je prvotno predstavila Polonius [7]. Delovanje bomo ponazorili na programu 10, vendar brez natančnih opisov relacij in množic, ki nastopajo pri dejanski analizi.

Program 10 ima poleg tipov predpisane še *regije* (angl. regions), ki si jih lahko predstavljamo kot življenjske dobe. Podrobneje so to množice *posoj* (angl. loans), ki jih kasneje natančno definiramo, za zdaj pa si jih lahko predstavljamo kot možne izvore (angl. origins) referenc (npr. `&x`, `&mut a.b` itd.). Reference so označene s številkami `'0`, `'1`, `'2` itd. Tukaj so prikazane kot del programa, vendar to ni veljavna sintaksa Rusta, je pa uporabna za razlago.

```

1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<'0 i32> = vec![];
4      let r: &'1 mut Vec<'2 i32> = &'3 mut v;
5      let p: &'5 i32 = &'4 x;
6      r.push(p);
7      x += 1;
8      take::<Vec<'6 i32>>(v);
9  }
10 fn take(p: T) { .. }

```

Program 10: Primer programa za Polonius iz [7]

Oglejmo si korake v programu 10, ki na koncu privedejo do napake:

- vrstica 3: Ustvarimo vektor deljenih referenc `v`.
- vrstica 4: Ustvarimo unikatno referenco `r`, ki kaže na vektor `v`.
- vrstici 5 in 6: Deljeno referenco na `x` vstavimo v vektor `v` preko `r`.
- vrstici 7 in 8: Poskušamo spremeniti vrednost `x`.

Vendar v vrstici 7 še vedno obstaja aktivna referenca na `x` v vektorju `v`, ki smo jo vstavili v vrstici 6. Vektor `v` še vedno potrebujemo v vrstici 8, zato javimo napako.

Oglejmo si, kako se intuitivno razumevanje napake prenese na analizo, ki jo opravi Polonius. Za lažje razumevanje si lahko predstavljamo, da algoritem trikrat obhodi kodo. To sicer ni povsem res, saj se ti obhodi v implementaciji prekrivajo, vendar je za razumevanje koristno.

Prvi obhod izračuna dva glavna elementa: vsebovanost regij med seboj in pripadnost posoj regijam.

Vsebovanost dveh regij se izračuna glede na pravila sklepanja Rustovega sistema tipov in jo zapišemo kot `'a: 'b`. To pomeni, da mora regija `'a` vsebovati vse posoje iz regije `'b`. Intuitivno mora referenca z življenjsko dobo `'a` živeti vsaj tako dolgo kot `'b`.

```
1 fn main() {  
2     let mut x: i32 = 22;  
3     let mut v: Vec<'0 i32> = vec![];  
4     let r: &'1 mut Vec<'2 i32> = &'3 mut v;  
5     // V zgornji vrstici se ustvari posoja L0 (iz &mut v), ki pripada  
6     // Vsebovanost - '3: '1, '2: '0, '0: '2  
7     let p: &'5 i32 = &'4 x;  
8     // Ustvarimo L1 (iz &x), ki pripada regiji '4  
9     // Vsebovanost - '4: '5  
10    r.push(p);  
11    // Vsebovanost - '5: '2  
12    x += 1;  
13    take::<Vec<'6 i32>>(v);  
14    // Vsebovanost - '0: '6  
15 }  
16 fn take(p: T) { .. }
```

Program 11: Primer programa za Polonius iz [7]

Pripadnost posoje regijam se določi ob ustvaritvi posoje. Posoje so interne strukture v Rustovem prevajalniku, ki hranijo podatke o ustvarjeni referenci [18]. V tem kontekstu pripadnost regiji pomeni, da se posoja zapiše kot dodaten metapodatek regije. Ko ustvarimo posojjo z `&` ali `&mut`, se tej določi pripadnost glede na regijo, ki je del tipa.

Za boljše razumevanje teh dveh korakov si oglejmo primer 11, kjer sta v komentarjih anotirana ta dva koraka.

i Zakaj v vrstici 6 programa 11 ustvarimo dvosmerno vsebovanost?

Če v vektor pišemo kot v vrstici 10, morajo elementi „znotraj“ reference živeti vsaj tako dolgo kot elementi v prvotnem vektorju. Zato dodamo vsebovanost `'2: '0`. Ker pa lahko iz vektorja tudi beremo, morajo elementi v prvotnem vektorju živeti vsaj tako

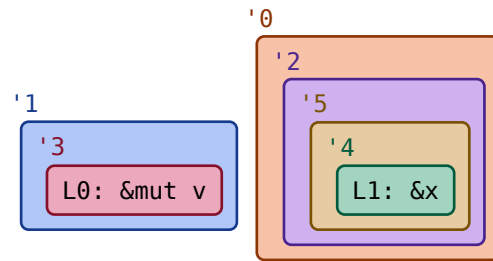


Diagram 1: Diagram vsebovanosti regij in posoj programa 11

dolgo kot tisti „znotraj“ reference, saj bi sicer lahko brali neveljaven pomnilnik. Tako dobimo še '0: '2.

Drugi obhod razširi vsebovanosti iz prvega obhoda, saj lahko nanje gledamo kot na relacijo matematične vsebovanosti, ki je tranzitivna. S tem dodeli posoje več regijam. Če sledimo tranzitivnemu zaprtju vsebovanosti, opazimo dve verigi:

- za posoj L0: '3: '1 in
- za posoj L1: '4: '5: '2: '0 ('0: '2 tukaj ni tako pomembno).

Osredotočimo se na posoj L1, ki je na koncu drugega obhoda pripadnica regije '0. Poleg razširitve vsebovanosti drugi obhod določi tudi aktivnost regij in posoj, vendar tega postopka tukaj ne bomo opisali. Povedali bomo le, da Polonius izračuna, da sta regija '0 in posledično posoja L1 aktivni v vrstici 12 v programu 11. Pojma aktivnosti regij in posoj sta tukaj analogna pojmu aktivnosti spremenljivk pri prevajalnikih.

V tretjem obhodu nato javimo napako, ker operacija spreminjanja vrednosti spremenljivke x v vrstici 12 v primeru 11 razveljavi pogoje posoje L1, ki je na tisti točki v programu še vedno živa. Razveljavitev pogojev posoje na kratko pomeni, da operacija ni dovoljena glede na tip reference, ki je ustvarila posoj. To so lahko na primer spreminjanje vrednosti mesta, na katero kaže deljena referenca, ali ustvarjanje nove reference na mesto, kjer že obstaja unikatna referenca.

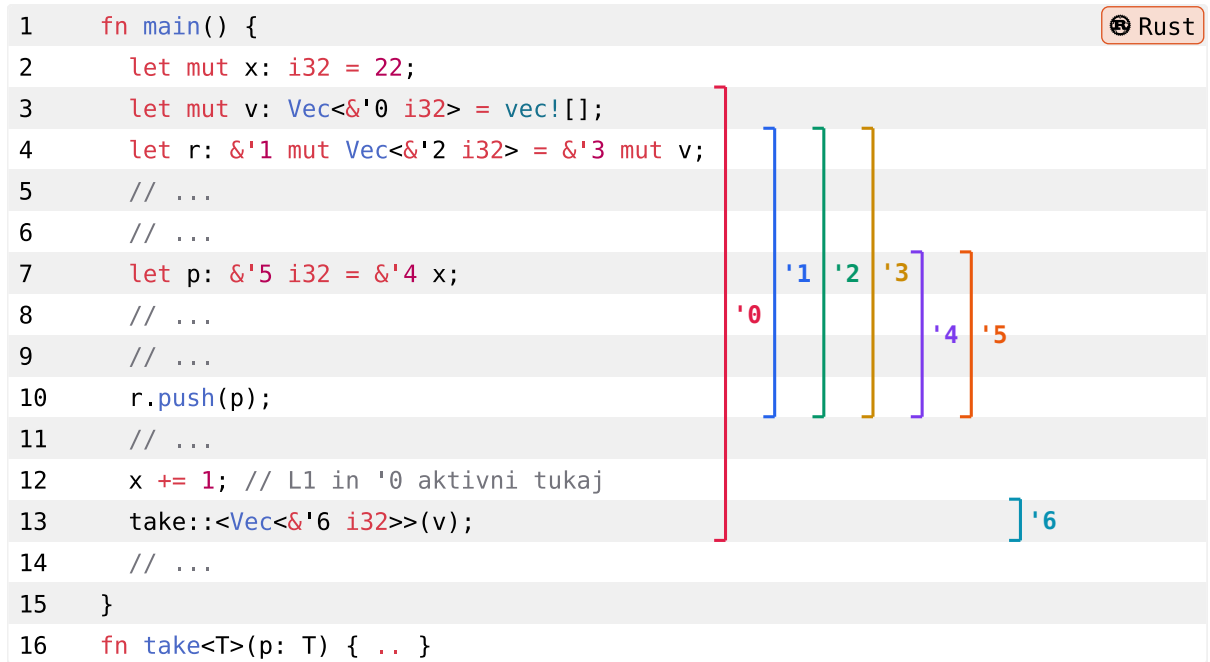


Diagram 2: Označene aktivnosti regij programa 11

V intuitivni razlagi smo izpustili številne podrobnosti, kot so izračun aktivnosti regij in posoj, podrobnosti razširitve različnih vsebovanosti skozi program ter pogoji za ustvarjanje drugih omejitev. Ob tem velja poudariti, da analiza deluje na MIRu, ki je v prevajalniku predstavljen kot graf, ne pa na samih vrsticah izvirne kode programa.

4.2 Formalizacija pravil

Cilj preverjevalnika izposoj je zadostiti pravilom lastništva. Ta pravila so običajno opisana intuitivno ali s primeri, kar je v prevajalniku težko formalno zajeti. Zaradi pomanjkanja uradne specifikacije se bomo oprli na delo Amande Stjerne [11], ki zastavi pet pravil v tabeli 1 in jih pojasni z razlago. To niso ista pravila, ki tvorijo osnovo za lastništvo, predstavljena v poglavju 3.

V tabeli 1 so podani pozitivni in negativni primeri za vsako pravilo, kot jih je predstavila Stjerna. Na podlagi teh primerov in njene razlage bomo formalno zapisali ta pravila z matematično notacijo. Vsa ta pravila delujejo na ravni posamezne funkcije, ne pa celotnega programa.

Pravilo Use-Init določa, da lahko uporabljamo samo mesta, ki so zagotovo inicializirana na točki v funkciji, kjer jih uporabljamo. Skupaj s praviloma Move-Deinit, ki pravi, da ne smemo uporabljati mest, katerih vrednost je bila premaknjena, ter Ref-Live, ki nam onemogoči dostop do sproščenih vrednosti preko referenc, tvori osnovo za sistem lastništva. Ta pravila nam na primer preprečijo vračanje vrednosti, ustvarjene na skladu, saj je ta na izhodu iz funkcije že sproščena [11].

Pravilo Unique-Write nam zagotavlja, da je lahko hkrati aktivna samo ena unikatna referenca ter pravilo Shared-Readonly podobno zagotavlja, da ne moremo pisati v oz. premikati iz mesta, na katerega kaže deljena referenca. Ti dve pravili nam omogočita, da prevajalnik zagotovi pravilno uporabo dveh vrst referenc, ki obstajata v Rustu.

Pri formalizaciji pravil bomo izhajali iz *grafa poteka* (angl. CFG - control flow graph), ki ga prevajalnik konstruira, še preden se začne faza preverjevalnika izposoj. Sestavljen je iz osnovnih blokov, ti pa iz stavkov. Vozlišča v samem grafu si lahko predstavljamo kot posamezne stavke, vendar jih kasneje v nalogi definiramo bolj podrobno.

Da lahko definiramo pravilo Use-Init, moramo uvesti še dve množici ter en predikat:

Poti(p) Množica Poti(p) poda vse poti skozi graf poteka od začetka funkcije do trenutne točke p v grafu poteka funkcije. Te poti so statične – ne spreminjajo se glede na vrednosti spremenljivk med izvajanjem programa. Predstavljamo si jih kot vse možne poti do trenutne točke ob poljubnih vhodnih vrednostih in spremenljivkah.

Uporabljenamesta(p) To je množica vseh mest, ki jih uporabimo na točki p . Uporaba je lahko branje iz mesta ali pisanje v mesto, uporaba

Pozitiven primer	Negativen primer
Use-Init	
<pre>let x: u32; if random() { x = 17; } else { x = 18; } let y = x + 1;</pre>	<pre>let x: u32; if random() { x = 17; } // ERROR: x not initialized: let y = x + 1;</pre>
Move-Deinit	
<pre>let tuple = (vec![1], vec![2]); moves_argument(tuple.1); // Does not overlap tuple.1: let x = tuple.0[0];</pre>	<pre>let tuple = (vec![1], vec![2]); moves_argument(tuple.0); // ERROR: use of moved value: let x = tuple.0[0];</pre>
Shared-Readonly	
<pre>struct Point(u32, u32); let mut pt = Point(13, 17); let x = &pt; let y = &pt; dummy_use(x); dummy_use(y);</pre>	<pre>struct Point(u32, u32); let mut pt = Point(13, 17); let x = &pt; // ERROR: assigned to // borrowed value: pt.0 += 1; dummy_use(x);</pre>
Unique-Write	
<pre>struct Point(u32, u32); let mut pt = Point(13, 17); let x = &mut pt; let y = &mut pt; //dummy_use(x); dummy_use(y);</pre>	<pre>struct Point(u32, u32); let mut pt = Point(13, 17); let x = &mut pt; // ERROR: cannot borrow `pt` // as mutable more than once: let y = &mut pt; dummy_use(x); dummy_use(y);</pre>
Ref-Live	
<pre>struct Point(u32, u32); let pt = Point(6, 9); let x = { &pt }; // pt still in scope let z = x.0;</pre>	<pre>struct Point(u32, u32); let x = { let pt = Point(6, 9); &pt }; // pt goes out of scope // ERROR: pt does not live // long enough: let z = x.0;</pre>

Tabela 1: Pravila preverjevalnika izposoj iz [11]. Pozitivni primeri predstavljajo mesta, kjer preverjevalnik sprejme kodo, negativni pa kjer jo zavrne.

projekcij (npr. komponent struktur), branje mesta preko reference itd. Bolj natančno jo definiramo s pomočjo interne strukture MIRa. Mesto se šteje kot uporabljeno, če nastopa kot operand ali ciljno mesto kjerkoli v stavku. Za podrobnejši pregled, kaj vse uporaba vključuje, si lahko ogledate unijo `StatementKind` v izvorni kodi prevajalnika [35].

Inicializirana(π, m, p) Predikat `Inicializirana( $\pi, m, p$ )` velja natanko tedaj, ko je mesto m skozi pot π inicializirano na točki p .

Pravilo `Use-Init` pravi, da za vsako točko p velja

$$\forall \pi \in \text{Poti}(p), m \in \text{UporabljenaMesta}(p) : \text{Inicializirana}(\pi, m, p)$$

Če obstaja točka p , za katero to ne velja, mora prevajalnik javiti napako. Podobno bomo brali tudi ostala pravila v tem razdelku.

Pred nadaljevanjem definiramo še pojem `predpone`, ki ga NLL RFC opiše tako [6]:

Predpona (angl. prefix) Pravimo, da so `predpone` leve vrednosti (angl. `lvalue`) vse tiste leve vrednosti, ki jih dobimo, če najprej odstranimo komponente ali dereferenciranja. Na primer, `predpone` leve vrednosti `*a.b` so `*a.b`, `a.b` in `a`.

Opomba o mestih in poteh premika (angl. move path)

Pojem `predpone` je načeloma definiran kot lastnost poti premika, ki je interni konstrukt prevajalnika. Vendar ga tukaj posplošimo na mesta iz MIRa, ker se bolje sklada z uporabljeno terminologijo. Rustov priročnik za prevajalnik tudi omenja, da sta ta pojma približno enaka. Pojem `predpone` uporabljamo namesto `spremenljivk`, ker nam lahko opiše gnezdene podatke, kot so komponente struktur.

Pravilo `Move-Deinit` nam prepereči, da uporabimo vezavo, iz katere je bila vrednost premaknjena. V kontekstu lastništva to pomeni, da ime ni več

lastnik vrednosti. Da pravilo definiramo formalno, moramo vpeljati še dva predikata.

Prekrivanje(m_1, m_2) Ta predikat pove, ali se mesti prekrivata, tj. ali je katero mesto predpona drugega. Velja natanko tedaj, ko je mesto m_1 predpona mesta m_2 ali obratno. Torej $\text{Prekrivanje}(\text{tuple.0}, \text{tuple.0.1})$ bi veljalo, $\text{Prekrivanje}(\text{tuple.0}, \text{tuple.1})$ pa ne.

Premaknjen(π, m, p) Predikat velja natanko tedaj, ko je bila vrednost iz mesta m premaknjena pred točko p na poti π . Premik vrednosti iz mesta v prevajalniku pomeni, da mesto m ni več v množici inicializiranih mest, torej ga prevajalnik iz nje odstrani. Intuitivno to pomeni, da mesto po premiku ni več inicializirano in ga ne moremo več uporabljati, dokler mu ne dodelimo nove vrednosti in posledično mesto dodamo nazaj v množico inicializiranih mest [36].

Torej pravilo Move-Deinit pravi, da mora za vsako točko p veljati

$$\begin{aligned} \exists \pi \in \text{Poti}(p), m_1 \in \text{UporabljenMesta}(p), m_2 : \\ \text{Prekrivanje}(m_1, m_2) \wedge \text{Premaknjen}(\pi, m_2, p) \end{aligned}$$

Z besedami povedano, pravilo Move-Deinit na neki točki p velja, ko ne obstaja nobena pot π do p , na kateri smo premaknili vrednost iz mesta m_2 , ki je predpona uporabljenega mesta m_1 .

Da bomo lahko razumeli naslednja pravila, moramo definirati pojem posoje, ki je tesno povezan s sorodnim pojmom „izraz izposoje“.

Izraz izposoje (angl. borrow expression) je jezikovni konstrukt, ki omogoča ustvarjanje reference (primer izraza izposoje je `&mut x`). Rustov priročnik za prevajalnik [33] pojma *borrow expression* ne definira, ampak ga uporablja tako:

„An Rvalue is an expression that creates a value: in this case, the rvalue is a mutable borrow expression, which looks like `&mut` “

V izvorni kodi prevajalnika je Rvalue definiran z unijo Rvalue [37]. Izraz izposoje natančno predstavlja varianta `Rvalue::ref(Region<'tcx>,`

`BorrowKind`, `Place<'tcx>`), ki ustvari referenco tipa `BorrowKind` na mesto `Place`.

Pojma izraza izposoje in posoje pogosto uporabljajo Weiss idr. v svojem članku o formalizaciji podmnožice Rusta [18]. Njihov način uporabe se sklada z našo definicijo, ki se glasi:

Posoja (angl. loan) Posoja je interni konstrukt prevajalnika, ki hrani podatke o referenci in njenem izvoru [18]. V trenutni implementaciji preverjevalnika izposoj je posoja predstavljena kot urejena trojica [6] $(\text{'a}, \text{shared|uniq|mut}, \text{lvalue})$, kjer velja naslednje:

- **'a**: Življenjska doba, za katero je vrednost izposojena. To se nanaša na življenjske dobe kot del Rustovega sistema tipov, ne pa na alternativno definicijo kasneje v nalogi, ki razume življenjske dobe kot množico posoj.
- **shared|uniq|mut**: To je tip posoje. Tipa posoje `uniq` in `mut` sta identična, vendar `uniq` ne pusti spreminjanja svojih referentov. Naša terminologija unikatne reference se sklada s tipom `mut`.
- **lvalue**: Leva vrednost, ki je bila izposojena.

V našem zapisu bomo torej posoje zapisovali kot $L = (\alpha, \tau, O)$, kjer bo $\tau \in \{\text{uniq}, \text{shrd}, \text{mut}\}$ predstavljal tip posoje, O pa levo vrednost (oziroma *izvor* z Rustovsko terminologijo).

Pravili Shared-Readonly in Unique-Write skrbita za veljavnost referenc in omejujeta njihovo uporabo. To sta isti pravili, ki smo ju opisali v razdelku 4.1. Pred opisom pravil moramo definirati še nekaj dodatnih predikatov.

PosojaAktivna(L, p) Predikat velja natanko tedaj, ko je posoja L aktivna na točki p . Intuitivno je posoja aktivna, kadar jo na tej točki še potrebuje neka aktivna regija. Natančno zvezo med regijami in posojami bomo opredelili v razdelku 4.6.2.

Poleg predikata za aktivnost posoje potrebujemo še predikate, ki opisujejo operacije nad mesti. V prevajalniku je takih tipov operacij več, vendar jih bomo zajeli v dve glavni vrsti.

RazveljaviDeljeno(m, p) Predikat velja natanko tedaj, ko se v točki p nad mestom m izvede operacija, ki bi lahko razveljavila deljeno posojjo mesta m (to bi bilo pisanje v mesto m ali pa ustvarjanje unikatne posoje).

RazveljaviUnikatno(m, p) Predikat velja, ko operacija razveljavi unikatno posojjo mesta m (ustvarjanje kakršnekoli nove posoje, pisanje v mesto, branje iz mesta).

Zdaj lahko sestavimo naslednji dve pravili. Pravilo Shared-Readonly pravi, da mora za vsako točko p veljati

$$\begin{aligned} \nexists L = (_, \tau, O), m : \\ \text{PosojaAktivna}(L, p) \wedge \tau = \text{shrd} \wedge \\ \text{Prekrivanje}(m, O) \wedge \text{RazveljaviDeljeno}(m, p) \end{aligned}$$

in pri pravilu Unique-Write mora veljati

$$\begin{aligned} \nexists L = (_, \tau, O), m : \\ \text{PosojaAktivna}(L, p) \wedge \tau \in \{\text{uniq}, \text{mut}\} \wedge \\ \text{Prekrivanje}(m, O) \wedge \text{RazveljaviUnikatno}(m, p) \end{aligned}$$

Pravilo Shared-Readonly na točki p torej velja, ko ne obstaja taka posoja $L = (_, \tau, O)$, ki je aktivna in katere izvor O se prekriva z mestom m , nad katerim smo ravno izvedli operacijo, ki bi razveljavila deljeno posojjo. Podobno razložimo pravilo Unique-Write.

Za zadnje pravilo potrebujemo še en predikat.

MestoAktivno(m, p) Predikat velja natanko tedaj, ko je mesto m še aktivno na točki p . Aktivnost mesta pomeni, da na tej točki v grafu poteka funkcije še ni bilo sproščeno (angl. dropped).

Potem mora za Ref-Live na vsaki točki p veljati

$$\begin{aligned} \nexists L = (_, _, O), m : \\ \text{PosojaAktivna}(L, p) \wedge \text{Prekrivanje}(m, O) \wedge \neg \text{MestoAktivno}(m, p) \end{aligned}$$

To preprosto pomeni, da morajo biti vse predpone izvora O aktivne posoje L tudi hkrati same aktivne.

4.3 Primer in formalizacija

V naslednjih razdelkih se bomo lotili glavnega dela naloge, matematične formalizacije delovanja Poloniusa. Da si bomo lažje predstavljali relacije in množice, bomo celotno delovanje ponazorili na programu 10 iz razdelka 4.1.

```

1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<&'0 i32> = vec![];
4      let r: &'1 mut Vec<&'2 i32> = &'3 mut v;
5      let p: &'5 i32 = &'4 x;
6      r.push(p);
7      x += 1;
8      take::<Vec<&'6 i32>>(v);
9  }
10 fn take(p: T) { .. }
```

Program 12: Primer programa za Polonius iz [7]. Ponovno prikazan program 10 za lažjo dostopnost.

4.4 Osnovne množice in elementi

Da lahko matematično govorimo o delovanju Poloniusa, moramo definirati osnovne množice in elemente, s katerimi bomo delali.

4.4.1 Množica posoj posoje

Množico vseh posoj označimo s *posoje*. *Pogoji posoje* so lastnosti, ki morajo držati v določeni točki programa, da posoj smatramo za veljavno oziroma aktivno. V literaturi ali izvorni kodi prevajalnika nikjer niso definirani

```

1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<'0 i32> = vec![];
4      let r: &'1 mut Vec<'2 i32> = &'3 mut v; // posoja L0 mesta `v`
5      let p: &'5 i32 = &'4 x; // posoja L1 mesta `x`
6      r.push(p);
7      x += 1;
8      take::<Vec<'6 i32>>(v);
9  }
10 fn take(p: T) { .. }

```

Program 13: Posoje v programu

eksplicitno, vendar se pa pojavi implicitna definicija preko razveljavitev pogojev posoje.

Razveljavitev pogojev posoje Pravimo, da *razveljavimo pogoje posoje*,

če velja ena izmed naslednjih točk:

- Referenca je deljena in
 - ustvarimo novo unikatno referenco *ali*
 - pišemo v mesto, ki je bilo izposojeno
- Referenca je unikatna in jo spreminjamo na kakršen koli način (ustvarjanje nove reference, pisanje, premikanje)

Zgornja pravila bolj formalno opisujejo pravila razveljavljanja posoje (angl. loan killed). Skratka, NLL RFC pravi, da za stavek na točki P v grafu definiramo „funkcijo prenosa“ – torej, katere posoje prinesemo v ali iz obsega [6]. Funkcija je definirana tako:

- *nekaj za nas nerelevantnih pravil*
- Če je stavek dodelitev $lv = \dots$, potem je vsaka posoja poti P, katere lv je predpona, razveljavljena.

V programu 13 vidimo, kako se posoje ustvarjajo tekom programa. Končamo z množico $posoje = \{L_0, L_1\}$.

4.4.2 Množica regij regije

V trenutni implementaciji preverjevalnika izposoj NLL se posoje spremljajo s pomočjo življenjskih dob. Tu smo življenjske dobe poimenovali regije (angl. regions). Množica regij je označena z *regije*. Na primeru so že označene z '1, '2, '3, itd. Pripadnost posoj regijam bomo kasneje določili z relacijo. Posamezno regijo si lahko predstavljamo kot množico posoj, vendar to ni povsem natančno, saj je tudi pripadnost posoje regiji odvisna od točke, je pa koristna intuitivna predstava.

4.4.3 Graf poteka in množica stavkov stavki

Graf poteka je že izračunan v prejšnjih fazah analize kode. Zgrajen je iz osnovnih blokov, ti pa iz stavkov. Množico vseh stavkov v MIRu označimo s *stavki*.

Graf spremlja tudi nekaj dodatnih metapodatkov. Ti so izračunani med analizo poteka podatkov (angl. dataflow analysis) [38]. Graf poteka označimo s $C = (\text{točke}, \text{povezave})$, kjer je *točke* množica vozlišč in *povezave* množica povezav. Uporabljamo izraz *točke* namesto vozlišča, ker se v literaturi večinoma uporablja izraz *point*, ne pa *node*.

Elementi množice *točke* so lahko dveh tipov:

- *na začetku stavka*: Označuje trenutek, preden se stavek izvede. Označimo jih s $S(\text{stmt})$.
- *med stavkom*: Označuje trenutek tik preden ima stavek učinek (v spletni objavi je napisano „just before the statement takes effect“). Označimo jih z $M(\text{stmt})$.

Avtor spletne objave pojmov *na začetku stavka* in *med stavkom* ne opredeli natančno, vendar lahko najdemo razlago v osnovni različici Poloniusa. Komentar nad strukturo, ki opisuje množico stavkov, pravi naslednje [39]:

„Ta struktura prevede MIR lokacijo, ki identificira stavek znotraj osnovnega bloka, v „obogateno lokacijo“, kar nam omogoči večjo granularnost.

```
1 fn primer(pogoj: bool) {  
2     let mut x = 1;  
3     if pogoj {  
4         x = 2;  
5     }  
6     let y = x;  
7 }
```

Program 14: Primer za graf poteka

Bolj podrobno, ločimo med začetkom in sredino stavka. Sredina stavka je točka *tik preden* ima stavek učinek. Torej za prirejanje $A = B$ bi bila sredina stavka trenutek ravno preden bi se B zapisal v A ...“

Poleg točk se ustvarijo še naslednje povezave:

- Za vsak stavek se ustvari povezava med njegovim začetkom in sredino:
 $\forall \text{ stmt} \in \text{stavki} : (S(\text{stmt}), M(\text{stmt})) \in \text{povezave}$
- Če $M(\text{stmt})$ predstavlja stavek na koncu bloka (angl. terminator), dodamo povezavo iz njega v $S(\text{stmt}')$ za vsak stavek stmt' na začetku ostalih osnovnih blokov, ki sledijo prvotnemu.

Opomba: To je zgolj matematična formulacija predstavitve grafa poteka; v prevajalniku je njegova predstavitev precej bolj kompleksna.

4.4.3.1 Primer grafa

Za lažjo predstavo grafa poteka konstruiramo graf za program 14 in njegov prevod v MIR (program 15). MIR bomo tukaj ponazorili s psevdokodo, vendar je sam MIR skupek struktur v Rustovem prevajalniku.

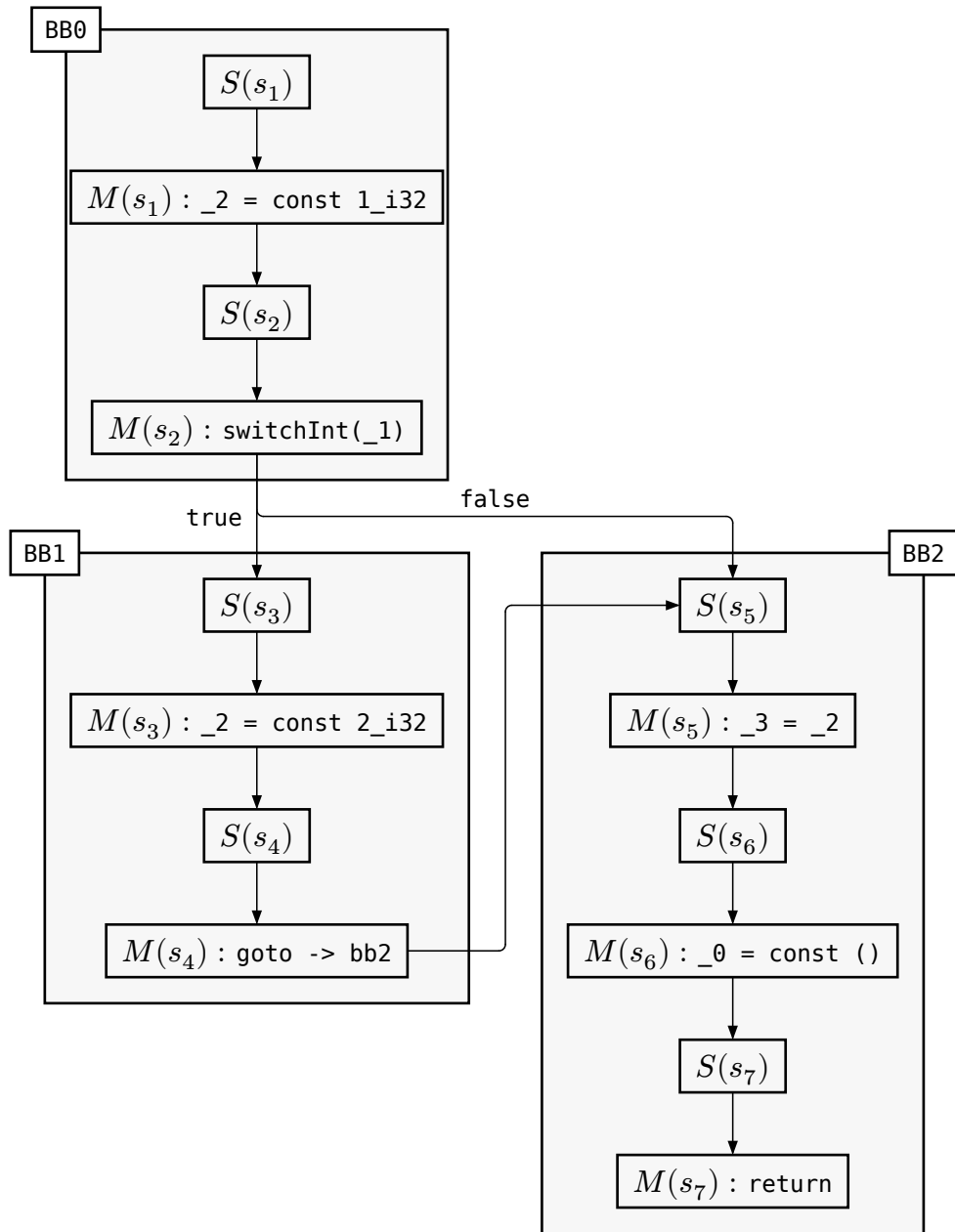
Konkretna sintaksa MIR je zasnovana izključno za pedagoške namene, zato se v njene podrobnosti ne bomo spuščali. Izpostavimo le naslednje:

- Spremenljivke izgubijo imena in se oštevilčijo ($_1$, $_2$, $_3$)
- Osnovni bloki so označeni z **bb**.
- `switchInt` je tip terminatorja, definiran v Rustovem prevajalniku [40].

S tem razumevanjem lahko zdaj program ponazorimo na sliki 3.


```
1  fn primer(_1: bool) -> () { Rust
2      let mut _0: ();          // povratna vrednost (unit type)
3      let mut _2: i32;         // x
4      let mut _3: i32;         // y
5
6      bb0: {
7          _2 = const 1_i32;    // let mut x = 1;
8          switchInt(_1) -> [
9              false: bb2,      // če je pogoj false -> skok na y = x
10             otherwise: bb1   // če je pogoj true -> skok na x = 2
11         ];
12     }
13
14     bb1: {
15         _2 = const 2_i32;    // x = 2;
16         goto -> bb2;
17     }
18
19     bb2: {
20         _3 = _2;              // let y = x;
21         _0 = const ();        // funkcija se konča (implicitni return ())
22         return;
23     }
24 }
```

Program 15: MIR za program 14



Slika 3: Graf poteka za program 14

4.5 Začetne relacije

V razdelku 4.4 smo definirali osnovne množice, nad katerimi bomo zdaj definirali relacije. Polonius je razdeljen na dva tipa relacij.

Začetne (angl. input) relacije so tiste, ki izhajajo že iz prejšnjih faz analize MIRa. Predstavljajo izhodiščno točko za celotno analizo in privzamemo, da so že izračunane. Iz njih nato izhajajo *izpeljane* relacije, ki so jedro Poloniusove analize.

4.5.1 Začetna relacija vsebovanosti

Začetno relacijo vsebovanosti (angl. base subset) označimo z `je_vsebovana_zacetno` $\subset$ regije $\times$ regije $\times$ točke. To je relacija, ki povezuje dve regiji na določeni točki v programu. Za intuicijo, zakaj je ta relacija pomembna, si lahko bralec ponovno prebere razdelek 4.1.

Natančneje, če velja $(R_1, R_2, P) \in \text{je_vsebovana_zacetno}$ pomeni, da je R_1 podmnožica regije R_2 na točki P v programu. Ker so regije konceptualno potenčne množice posoj na posamezni točki, si lahko relacijo razložimo tako, da regija R_2 vsebuje vse posoje, ki jih vsebuje R_1 , zato R_2 inducira več omejitev na uporabi mest, ki so izposojena. Relacija mora veljati na sredini stavka ($M(\text{stmt})$), ki inducira njen nastanek. Na primer, na sredini stavka `let a: &'1 i32 = &'2 b;` zapišemo $(\text{'2}, \text{'1}, P) \in \text{je_vsebovana_zacetno}$.

Opomba: V primerih programov bomo uporabljali oznako $<:$, ki predstavlja vsebovanost med tipi (angl. subtyping relation).

Povezava z NLL

V NLL so regije predstavljene kot množice točk oziroma stavkov, kjer je vrednost, ki vsebuje regijo v svojem tipu, veljavna. Torej `'a: 'b` pomeni, da mora biti `'a` veljavna vsaj toliko časa kot `'b`. V angleščini bi temu rekli *'a preživi* (angl. outlives) *'b*. Drugače povedano, množica točk `'b` bi bila podmnožica `'a`, kar pa je

```
1 fn main() {
2     let mut x: i32 = 22;
3     let mut v: Vec<&'0 i32> = vec![];
4
5     let r: &'1 mut Vec<&'2 i32> = &'3 mut v;
6     // tukaj zahtevamo naslednje: &'3 mut Vec<&'0 i32> <: &'1 mut
7     // Vec<&'2 i32>
8     // ('3, '1, P) ∈ je_vsebovana_zacetno, ('0, '2, P) ∈
9     // je_vsebovana_zacetno, ('2, '0, P) ∈ je_vsebovana_zacetno
10
11     let p: &'5 i32 = &'4 x;
12     // zahtevamo: &'4 i32 <: &'5 i32
13     // ('4, '5, P) ∈ je_vsebovana_zacetno
14
15     r.push(p);
16     // zahtevamo: &'5 i32 <: &'2 i32
17     // ('5, '2, P) ∈ je_vsebovana_zacetno
18
19     x += 1;
20
21     take::<Vec<&'6 i32>>(v);
22     // zahtevamo Vec<&'0 i32> <: Vec<&'6 i32>
23     // ('0, '6, P) ∈ je_vsebovana_zacetno
24 }
```

Program 16: Začetna relacija vsebovanosti

ravno obratno zapisano kot v Poloniusu. Ključna razlika je, da so regije v Poloniusu množice posoj, ne pa točk. Intuitivno lahko rečemo, da vsaka nova posoja prinese dodatne omejitve k uporabi in ustvarjanju referenc. Zato je smiselno, da je v Poloniusu regija 'a podmnožica regije 'b, saj potem mora upoštevati isto ali manj omejitev kot 'b.

```

1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<'0 i32> = vec![];
4      let r: &'1 mut Vec<'2 i32> = &'3 mut v;
5      // ('3, L0, P) ∈ regija_posojena
6      let p: &'5 i32 = &'4 x;
7      // ('4, L1, P) ∈ regija_posojena
8      r.push(p);
9      x += 1;
10     take:::<Vec<'6 i32>>(v);
11 }

```

Program 17: Začetna relacija posoje regij

4.5.2 Začetna relacija posoje regij

Začetno relacijo posoje regij (angl. borrow region) označimo z $\text{regija_posojena} \subseteq \text{regije} \times \text{posoje} \times \text{točke}$.

Če velja $(R, L, P) \in \text{regija_posojena}$, pomeni, da izraz izposoje na točki P ustvari posojjo L in postane del regije R . Tako kot relacija vsebovanosti se tudi ta zahteva vzpostavi na sredini stavka.

To je ključna relacija, ki poveže regije, ki so del Rustovih tipov, in posoje, ki so metapodatki v Rustovem prevajalniku. S pomočjo te relacije lahko povežemo določene reference z regijami, sledimo, kje so aktivne tekom programa, in ugotovimo, kdaj javimo napako.

Z relacijama $\text{je_vsebovana_zacetno}$ in regija_posojena lahko sestavimo diagram vsebovanosti 4. Pričakovali bi, da bi veljala tranzitivnost, kot pri vsebovanosti v teoriji množic, vendar gre le za začetna dejstva, zato se druge lastnosti upoštevajo kasneje v analizi.

4.5.3 Relacija aktivnosti regije

Začetno relacijo aktivnosti regije (angl. region live at) označimo z $\text{regija_aktivna_na} \subseteq \text{regije} \times \text{točke}$. Relacija $(R, P) \in \text{regija_aktivna_na}$

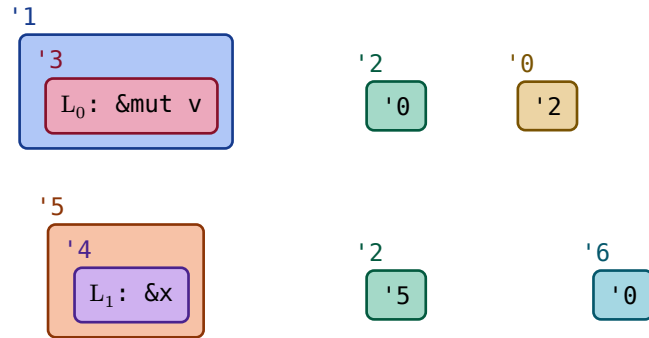


Diagram 4: Vsebovanosti za program 17

pomeni, da je regija R aktivna na točki P . Torej bo spremenljivka, katere tip vključuje R (na primer $\&a$), morda kasneje v programu uporabljena.

To določi analiza aktivnosti, ki poteka enako kot v NLL RFC. Bolj natančno, s pomočjo različnih omejitev izračuna množico točk, kjer mora biti regija (v RFC-ju imenovana *lifetime*) aktivna [6].

To relacijo smo prikazali na diagramu 2 v poglavju 4.1, ki si ga je priporočljivo še enkrat ogledati za boljšo intuicijo o tem, kaj aktivnost pomeni.

4.5.4 Relacija prekinitve posoje

Začetno relacijo prekinitve posoje (angl. loan killed at) označimo s $\text{posoja_prekinjena_na} \subseteq \text{posoje} \times \text{točke}$. $(L, P) \in \text{posoja_prekinjena_na}$ pomeni, da je posoja L prekinjena (angl. killed) na točki P . Pojem prekinitve oziroma razveljavitve pogojev smo že definirali v razdelku 4.4.1. To se običajno zgodi na sredini prireditvenega stavka, ki prepíše mesto, prej povezano s posojom L .

V našem primeru nimamo primera prekinitve posoje, zato njeno ključnost ponazorimo v programu 18.

```

1 let p = 22;
2 let q = 44;
3 let x: &mut i32 = &mut p; // `x` kaže na `p`
4 let y = &mut *x; // Posoja  $L_0$ , `y` kaže tudi na `p`
5 // ... vmes nikjer ne uporabimo `x`
6 x = &mut q; // `x` kaže na `q`; prekine  $L_0$ 
7 // Lahko uporabimo *x tukaj

```

 Rust

Program 18: Primer prekinitve posoje

V tem programu x kaže na p , y pa si sposodi isto mesto prek x (posoja L_0). Dokler je posoja L_0 aktivna, ne moremo spreminjati vrednosti p prek $*x$, saj bi s tem razveljavili pogoje posoje. Ko pa x priredimo novo vrednost, prekinemo posojjo L_0 in si s tem znova omogočimo dostop do $*x$. Brez prekinitve bi Polonius menil, da je mesto $*x$ še vedno izposojeno, čeprav zdaj y kaže na p in x na q .

4.5.5 Relacija razveljavitve posoje

Začetno relacijo razveljavitve posoje (angl. invalidates loan) označimo s $\text{posoja_razveljavljena_na} \subset \text{točke} \times \text{posoje}$. To pomeni, da dejanje na točki P (na primer spreminjanje izposojenega mesta) razveljavi pogoje posoje L , kar je že opisano v poglavju o definiciji množice posoje .

4.6 Izpeljane relacije

V tem poglavju bomo opisali relacije, ki jih izpeljemo iz začetnih. V primerih pri relacijah ne bomo označevali točk v grafu poteka, ker bodo relacije pripisane na tistem mestu v programu, kjer posamezna relacija velja. V ozadju se analiza še vedno izvaja na nivoju MIRa, vendar za naše poenostavljene primere to ni bistveno. Torej bomo pisali $(R_1, R_2) \in \text{je_vsebovana_zacetno}$ namesto $(R_1, R_2, P) \in \text{je_vsebovana_zacetno}$.

```

1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<'0 i32> = vec![];
4      let r: &'1 mut Vec<'2 i32> = &'3 mut v;
5      // ('3, L0, P) ∈ regija_posojena
6      let p: &'5 i32 = &'4 x;
7      // ('4, L1, P) ∈ regija_posojena
8      r.push(p);
9      x += 1; // tukaj razveljavimo L1 s spreminjanjem deljenega
              referenta
10     take::<Vec<'6 i32>>(v);
11 }

```

Program 19: Primer razveljavitve posoje

4.6.1 Relacija vsebovanosti

Začetno relacijo vsebovanosti razširimo v relacijo vsebovanosti (angl. subset), ki jo označimo z $je_vsebovana \subseteq regije \times regije \times točke$. Definirana je z naslednjimi pravili:

1. **Začetna relacija:** Če velja $(R_1, R_2, P) \in je_vsebovana_zacetno$, potem velja $(R_1, R_2, P) \in je_vsebovana$. Torej se vse trojice iz začetne relacije pojavijo tudi v razširjeni.
2. **Tranzitivnost:** Če veljata $(R_1, R_2, P) \in je_vsebovana$ in $(R_2, R_3, P) \in je_vsebovana$, potem sledi $(R_1, R_3, P) \in je_vsebovana$. Relacija vsebovanosti na isti točki v programu je tranzitivna.
3. **Propagacija:** Če velja
 1. $(R_1, R_2, P) \in je_vsebovana$ in
 2. $(P, Q) \in povezave$: (točki si sledita v grafu poteka) in
 3. $(R_1, Q) \in regija_aktivna_na$: (regija 1 je aktivna na naslednji točki) in
 4. $(R_2, Q) \in regija_aktivna_na$: (regija 2 je aktivna na naslednji točki),


```

1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<'0 i32> = vec![];
4
5      let r: &'1 mut Vec<'2 i32> = &'3 mut v;
6      // ('3, '1) in je_vsebovana, ('0, '2) ∈ je_vsebovana, ('2, '0) ∈
        je_vsebovana
7
8      let p: &'5 i32 = &'4 x;
9      // ('3, '1) ∈ je_vsebovana, ('0, '2) ∈ je_vsebovana, ('2, '0) ∈
        je_vsebovana
10     // ('4, '5) ∈ je_vsebovana
11
12     r.push(p);
13     // ('3, '1) ∈ je_vsebovana, ('0, '2) ∈ je_vsebovana, ('2, '0) ∈
        je_vsebovana
14     // ('4, '5) ∈ je_vsebovana
15     // ('5, '2) ∈ je_vsebovana
16
17     x += 1;
18
19     take::<Vec<'6 i32>>(v);
20     // ('3, '1) ∈ je_vsebovana, ('0, '2) ∈ je_vsebovana, ('2, '0) ∈
        je_vsebovana
21     // ('4, '5) ∈ je_vsebovana
22     // ('5, '2) ∈ je_vsebovana
23     // ('0, '6) ∈ je_vsebovana
24 }
25
26 fn take(p: T) { .. }

```

Program 20: Relacija vsebovanosti

tedaj velja $(R_1, R_2, Q) \in \text{je_vsebovana}$. To pomeni, da se relacija propa-
gira čez graf poteka, če sta obe regiji aktivni na naslednji točki v grafu.

Pogoj za aktivnost nam pride prav kasneje.

Primeru pripišemo te relacije v programu 20.

4.6.2 Relacija zahteve

Relacija zahteve nam pove, da regija R zahteva, da pogoji posoje L veljajo na točki P . Označimo jo z $zahteva \subseteq \text{regije} \times \text{posoje} \times \text{točke}$ in je definirana z naslednjimi pravili:

1. **Začetna relacija:** Če velja $(R, L, P) \in \text{regija_posojena}$, potem velja $(R, L, P) \in \text{zahteva}$. To nam pove, da če se trojica nahaja v relaciji posoje regij, se nahaja tudi v relaciji zahteve.
2. **Vsebovanost:** Če veljata $(R_1, L, P) \in \text{zahteva}$ in $(R_1, R_2, P) \in \text{je_vsebovana}$, potem sledi $(R_2, L, P) \in \text{zahteva}$. To nam pove, da če neka regija R_1 , ki je podmnožica večje regije R_2 , na točki P zahteva posoj L , potem tudi R_2 zahteva isto posoj.
3. **Propagacija:** Če velja
 1. $(R, L, P) \in \text{zahteva}$: (regija R zahteva posoj L na P) in
 2. $(L, P) \notin \text{posoja_prekinjena_na}$: (posoja L ni prekinjena na P) in
 3. $(P, Q) \in \text{povezave}$: (točka Q sledi P v grafu poteka) in
 4. $(R, Q) \in \text{regija_aktivna_na}$: (regija R je aktivna na točki Q),
 potem sledi $(R, L, Q) \in \text{zahteva}$.

Opazimo, da pri relaciji vsebovanosti $\text{je_vsebovana_zacetno}$ in pri relaciji zahteve zahteva mora biti regija pri pravilu za propagacijo aktivna na naslednji točki Q . S programom 21 ponazorimo zakaj je to pomembna omejitev.

4.6.3 Relacija aktivnosti posoje

Relacija aktivnosti posoje (angl. loan live at) pomeni, da je posoja L aktivna na točki P . Označimo jo s $\text{posoja_aktivna_na} \subseteq \text{posoje} \times \text{točke}$ in velja

$$(L, P) \in \text{posoja_aktivna_na} \iff$$

$$\exists R \in \text{regije} : (R, P) \in \text{regija_aktivna_na} \wedge (R, L, P) \in \text{zahteva}$$

To pomeni, da je posoja aktivna, natanko tedaj, ko jo na točki P zahteva neka aktivna regija.

```

1  let x = 22;
2  let y = 44;
3
4  let mut p: &'0 i32 = &'1 x; // posoja L0
5  // ('1, '0) ∈ je_vsebovana
6  // ('1, L0) ∈ zahteva
7
8  p = &'3 y; // posoja L1
9  // ('3, '0) ∈ je_vsebovana
10 // ('3, L1) ∈ zahteva
11 // '1 ni več aktivna, ker smo jo prepisali z '3
12
13 x += 1;
14 // Razveljavi se posoja L0: (L0) ∈ posoja_razveljavljena_na
15 // Tukaj bi brez pravila o aktivnosti regij še vedno zahtevali (L0,
16 // '0) ∈ zahteva zaradi pravila o propagaciji
17 print(*p);
18 // Ta izraz je tukaj, da je referenca `p` še vedno živa pri izrazu
19 x += 1, sicer bi Rustov prevajalnik takoj že zavrgel (angl. drop)
20 spremenljivko `p` po vrstici 8.

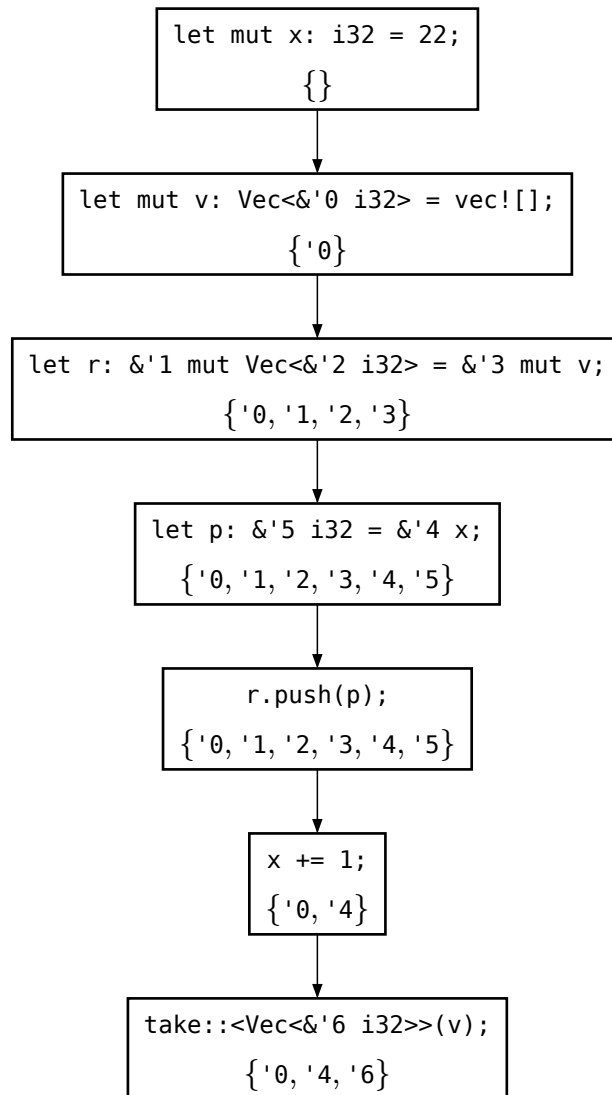
```

Program 21: Primer relacije zahteve

4.6.4 Vizualizacija na primeru

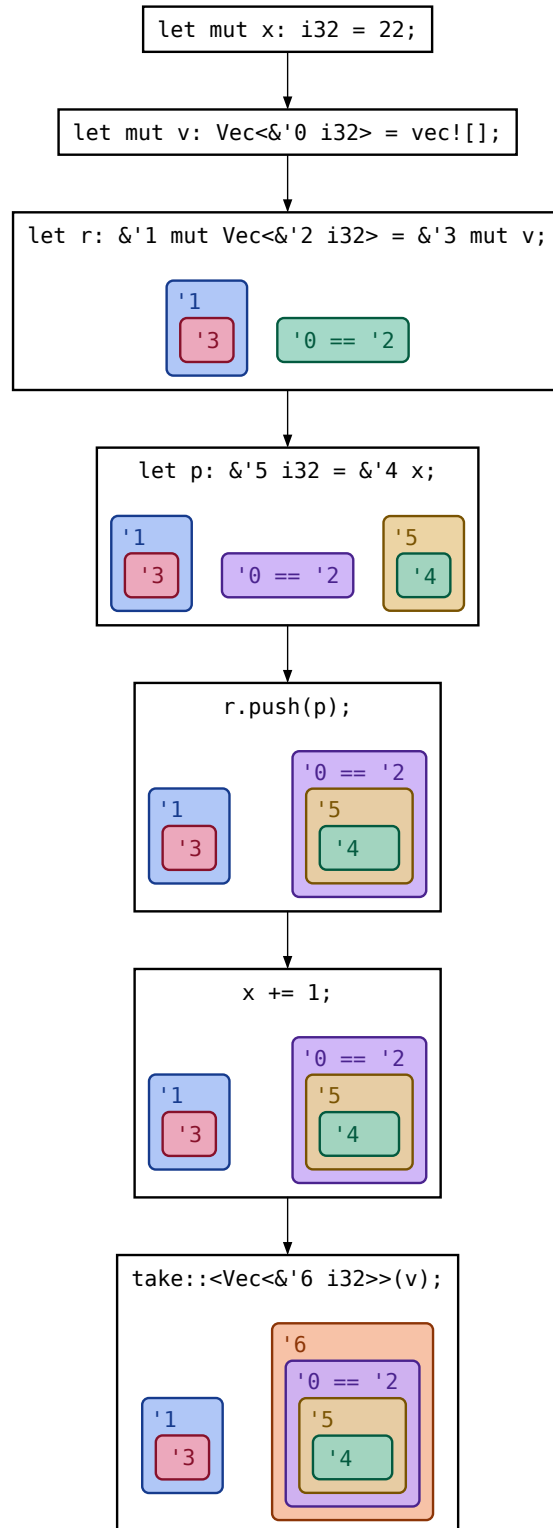
Za lažjo predstavo v tem razdelku vizualiziramo glavne relacije na primeru. Najprej na vsaki točki (oz. vrstici v našem poenostavljenem primeru) določimo množico vseh aktivnih regij. To nam poda relacija `regija_aktivna_na` in je prikazana na diagramu 5.

Nato vizualno ponazorimo razširjeno relacijo `je_vsebovana`, ki že upošteva pravila tranzitivnosti in propagacije. To prikazuje diagram 6. Iz diagrama za `je_vsebovana` konstruiramo diagram 7 za `zahteva`, tako da sledimo pravilom, opisanim v razdelku 4.6.2.

Diagram 5: Relacija `regija_aktivna_na`

4.7 Javljanje napake

S pomočjo prejšnjih relacij lahko na koncu definiramo, kje v programu javimo napako (v obsegu preverjevalnika posoj). Ponovno si pomagamo z relacijo, ki jo tokrat poimenujemo *relacija napake* (angl. error) in jo

Diagram 6: Relacija `je_vsebovana`

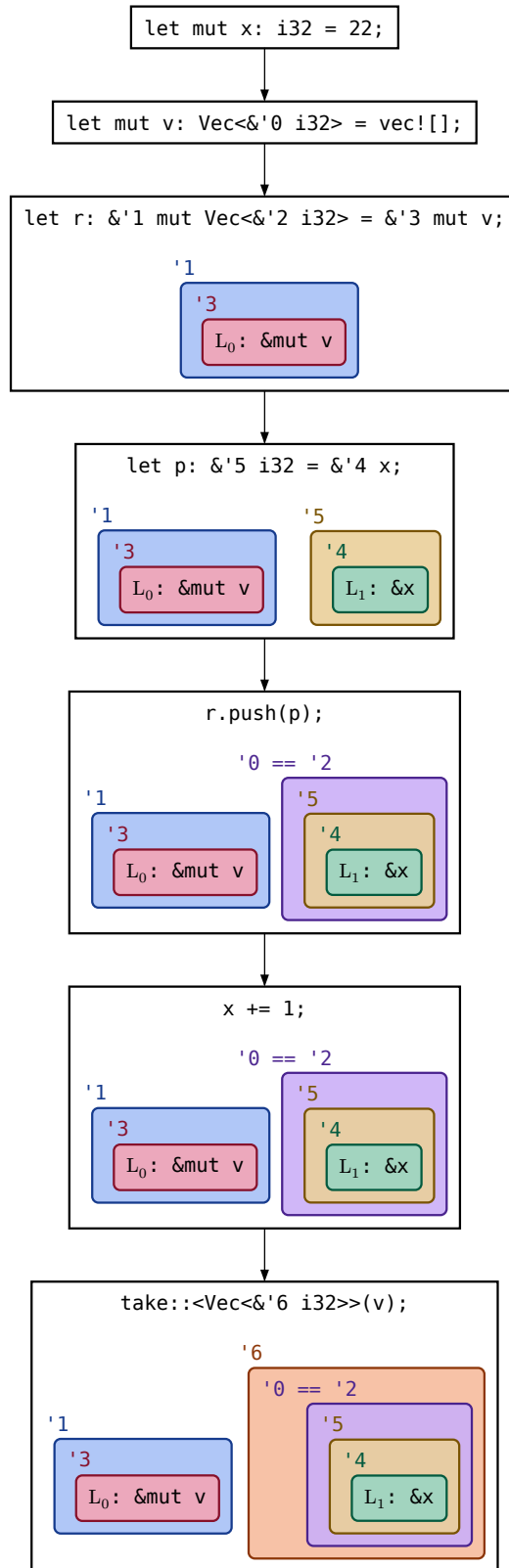


Diagram 7: Relacija zahteva

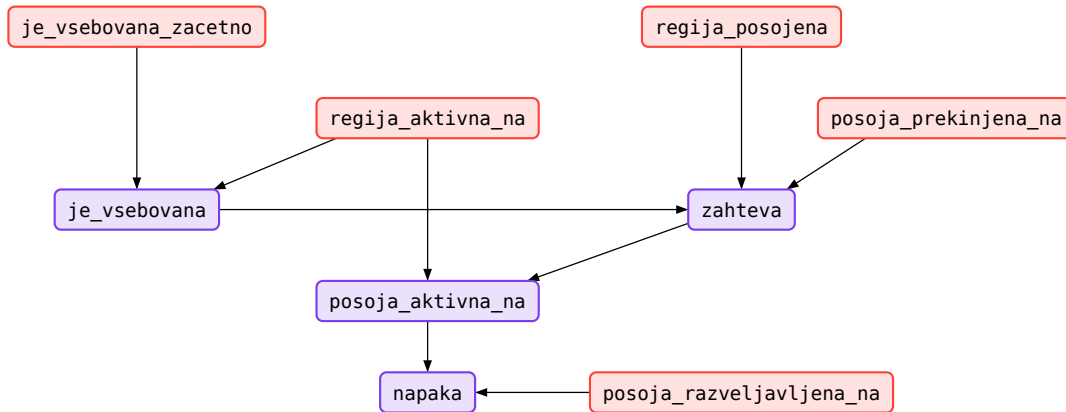


Diagram 8: Relacije Poloniusa. Z rdečo so označene začetne relacije, z vijolično pa izpeljane.

označimo z *napaka*. Ta relacija nam pove, da javimo napako na točki *P* v programu in velja

$$P \in \text{napaka} \iff$$

$$\exists L \in \text{posoje} : (P, L) \in \text{posoja_razveljavljena_na} \wedge (L, P) \in \text{posoja_aktivna_na}$$

Napaka se torej javi natanko tedaj, ko neko dejanje na točki *P* razveljavi pogoje posoje *L*, ki je hkrati tudi aktivna na točki *P*.

Poglejmo še, kako se napaka dokončno javi na programu 22. Če je kakšen korak nejasen, si lahko pomagata z diagrami v prejšnjem poglavju.

4.8 Vizualna reprezentacija delovanja Poloniusa

Da si lažje predstavljamo, kako se različne relacije povezujejo, bomo v tem razdelku prikazali diagram 8, ki prikazuje vse relacije in povezave med njimi. Diagram je zelo podoben tistemu iz [11], vendar poenostavljen, saj se naša naloga ukvarja samo z bistvom Poloniusa in ne z njegovo implementacijo. Puščica, ki kaže iz prve relacije v drugo, pomeni, da se za izpeljavo druge relacije zanašamo na prvo.

```
1  fn main() {
2      let mut x: i32 = 22;
3      let mut v: Vec<&'0 i32> = vec![];
4
5      let r: &'1 mut Vec<&'2 i32> = &'3 mut v;
6      // relacije, ki so ustvarjene tukaj niso relevantne za napako
7
8      let p: &'5 i32 = &'4 x;
9      // ('4, L1) ∈ `zahteva`
10     // ('4, '5) ∈ `je_vsebovana` ⇒ ('5, L1) ∈ `zahteva`
11
12     r.push(p);
13     // ('5, '2) ∈ `je_vsebovana` ⇒ ('2, L1) ∈ `zahteva`
14
15     x += 1;
16     // Tukaj se razveljavi posoja L1: (L1) ∈
17     // `posoja_razveljavljena_na`.
18     // Da se javi napaka, mora biti ta posoja aktivna (L1) ∈
19     // `posoja_aktivna_na`.
20     // Torej jo mora zahtevati neka aktivna regija, na trenutni točki
21     // pa je aktivna regija '2, ker jo lahko uporabimo v funkciji
22     // `take`, ki sprejme naš vektor `v`. Elementi vektorja imajo regijo
23     // '2, ki zahteva posoj L1.
24     // Torej, ker smo razveljavili posoj L1, medtem ko je bila
25     // aktivna regija, ki jo ta posoja zahteva, javimo napako.
26
27     take::<Vec<&'6 i32>>(v);
28 }
29
30 fn take(p: T) { .. }
```

Program 22: Napaka v programu

Poglavje 5

Zaključek

Ena izmed glavnih prednosti Rusta je njegovo „brezplačno“ (angl. zero-cost) upravljanje pomnilnika med izvajanjem programa. To sicer zahteva več časa pri prevajanju zaradi preverjevalnika izposoj, ki določa, kaj velja za pomnilniško varno in kaj se zavrne.

Trenutna implementacija preverjevalnika izposoj NLL je v nekaterih primerih preveč konzervativna in posledično zavrne varne programe, ki bi jih lahko sprejeli z bolj natančno analizo [6]. Zato je N. D. Matsakis v svoji spletni objavi opisal Polonius, ki bolje sledi toku podatkov v programu in lahko sprejme te bolj kompleksne primere [7].

V nasprotju z NLL, ki je formalno definiran znotraj RFC dokumenta [6], je bila Poloniusova definicija od začetka neformalna in prepletena z implementacijo. Polonius je bil napisan v Datalogu, ki je podmnožica Prologa, nato pa v Rustu. Ekipa, ki ga je implementirala, se nikoli ni ukvarjala s točnim opisom njegovega delovanja in do pred kratkim je bil eden redkih virov formalne specifikacije magistrska naloga Amande Stjerne [11], ki je ena izmed razvijalcev Rusta. Šele v zadnjem letu se je pojavil projekt `amir-formality`, ki želi sestaviti uradno specifikacijo za Rustov sistem tipov in preverjevalnik izposoj (vključno s Poloniusom) [30].

Cilj te naloge je bil formulirati alternativo Datalog implementaciji Poloniusa na formalen matematičen način, da bi omogočili lažje razumevanje tega kompleksnega sistema. Datalog implementacija Poloniusa je bila prvotna formulacija Poloniusa v [41] in je idejno sledila prvotni spletni objavi N.D. Matsakisa. Kasneje se je Polonius premaknil v glavno vejo Rustovega prevajalnika in trenutno se v njem nahaja v razvojni obliki, ki zamenja nekaj moči preverjanja pravilnosti za hitrost prevajanja. Naša formulacija se v načinu delovanja ujema z Datalog implementacijo, vendar je za razumevanje nekoliko poenostavljena.

Formulacijo smo oblikovali s pomočjo množic in relacij, definiranih nad njimi. Osnovne množice so predstavljale Rustove strukture v prevajalniku, s pomočjo katerih se definirajo začetne relacije, ki so dejstva, iz katerih izhaja celotna analiza.

Te smo nadgradili z izpeljanimi relacijami, ki tvorijo jedro Poloniusovega delovanja. Začetna dejstva smo s pomočjo pravil o tranzitivnosti, aktivnosti ter propagaciji skozi graf razširili v končne relacije, s katerimi smo nazadnje definirali relacijo napake. Ta končna relacija velja natanko tedaj, ko bi na tisti točki v programu javili napako.

Lahko bi rekli, da je ta formalizacija odveč, saj že Datalog pravila formalno definirajo delovanje Poloniusa. A če pogledamo izvirno kodo implementacije, vidimo, da je že začetnih relacij osemnajst [42]. Naša formalizacija poda poenostavljen način razumevanja delovanja algoritma z matematičnega vidika. Sicer ni dovolj močna, da bi lahko z njo dokazovali izreke ali leme, vendar nam poda trdno osnovo, iz katere lahko gradimo razumevanje Rustovega preverjevalnika izposoj.

Literatura

- [1] „Microsoft: 70 Percent of All Security Bugs Are Memory Safety Issues | ZDNET“, <https://www.zdnet.com/article/microsoft-70-percent-of-all-security-bugs-are-memory-safety-issues/>, dostopano avg. 17, 2025.
- [2] „Memory Safety“, <https://www.chromium.org/Home/chromium-security/memory-safety/>, dostopano avg. 15, 2025.
- [3] M. Glazar, „Is Coding in Rust as Bad as in C++?“, <https://quick-lint-js.com/blog/cpp-vs-rust-build-times/>, dostopano avg. 18, 2025.
- [4] H. G. Baker, „List Processing in Real Time on a Serial Computer“, *Communications of the ACM*, let. 21, št. 4, str. 280–294, apr. 1978.
- [5] S. Klabnik in C. Nichols, *The Rust Programming Language*, 2nd edition, No Starch Press, San Francisco, 2023.
- [6] „2094-Nll - The Rust RFC Book“, <https://rust-lang.github.io/rfcs/2094-nll.html>, dostopano jan. 13, 2025.
- [7] N. D. Matsakis, „An Alias-Based Formulation of the Borrow Checker · Baby Steps“, <https://smallcultfollowing.com/babysteps/blog/2018/04/27/an-alias-based-formulation-of-the-borrow-checker/>, dostopano jan. 2, 2025.
- [8] „Polonius Revisited, Part 2 · Baby Steps“, <https://smallcultfollowing.com/babysteps/blog/2023/09/29/polonius-part-2/>, dostopano avg. 20, 2025.
- [9] „Polonius Revisited, Part 1 · Baby Steps“, <https://smallcultfollowing.com/babysteps/blog/2023/09/22/polonius-part-1/>, dostopano mar. 1, 2025.
- [10] „What Is Polonius? - Polonius“, <https://rust-lang.github.io/polonius/>, dostopano avg. 20, 2025.

- [11] A. Stjerna, „Modelling Rust’s Reference Ownership Analysis Declaratively in Datalog“, 2020.
- [12] „The First Six Years in the Development of Polonius - Amanda Stjerna | EuroRust 2024“, https://www.youtube.com/watch?v=uCN_LRcswts, dostopano nov. 1, 2025.
- [13] „4 Years of Rust | Rust Blog“, <https://blog.rust-lang.org/2019/05/15/4-Years-Of-Rust/>, dostopano jan. 20, 2026.
- [14] Y. Matsushita, X. Denis, J.-H. Jourdan, in D. Dreyer, „RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code“, *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, ACM, San Diego CA USA, 2022, str. 841–856.
- [15] N. Villani, J. Hostert, D. Dreyer, in R. Jung, „Tree Borrows“, *Proceedings of the ACM on Programming Languages*, let. 9, št. PLDI, str. 1019–1042, jun. 2025.
- [16] B. C. Pierce, *Types and Programming Languages*, The MIT Press, Cambridge, Massachusetts London, England, 2002.
- [17] R. Jung, J.-H. Jourdan, R. Krebbers, in D. Dreyer, „RustBelt: Securing the Foundations of the Rust Programming Language“, *Proceedings of the ACM on Programming Languages*, let. 2, št. POPL, str. 1–34, jan. 2018.
- [18] A. Weiss, O. Gierczak, D. Patterson, in A. Ahmed, „Oxide: The Essence of Rust“, <https://arxiv.org/abs/1903.00982>, dostopano maj 26, 2025.
- [19] W. Crichton, G. Gray, in S. Krishnamurthi, „A Grounded Conceptual Model for Ownership Types in Rust“, *Proceedings of the ACM on Programming Languages*, let. 7, št. OOPSLA2, str. 1224–1252, okt. 2023.

- [20] E. Reed, „Patina: A Formalization of the Rust Programming Language“.
- [21] F. Wang, F. Song, M. Zhang, X. Zhu, in J. Zhang, „KRust: A Formal Executable Semantics of Rust“, *2018 International Symposium on Theoretical Aspects of Software Engineering (TASE)*, 2018 International Symposium on Theoretical Aspects of Software Engineering (TASE), 2018, str. 44–51.
- [22] D. J. Pearce, „A Lightweight Formalism for Reference Lifetimes and Borrowing in Rust“, *ACM Transactions on Programming Languages and Systems*, let. 43, št. 1, str. 1–73, mar. 2021.
- [23] S. Ho, A. Fromherz, in J. Protzenko, „Sound Borrow-Checking for Rust via Symbolic Semantics (Long Version)“, <http://arxiv.org/abs/2404.02680>, dostopano maj 15, 2025.
- [24] M. Tofte in J.-P. Talpin, „Region-Based Memory Management“, *Information and Computation*, let. 132, št. 2, str. 109–176, feb. 1997.
- [25] D. Grossman, G. Morrisett, T. Jim, M. Hicks, Y. Wang, in J. Cheney, „Region-Based Memory Management in Cyclone“.
- [26] M. Fluet, G. Morrisett, in A. Ahmed, „Linear Regions Are All You Need“, *Programming Languages and Systems*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006.
- [27] „Rust-Lang/Polonius“, <https://github.com/rust-lang/polonius>, dostopano nov. 6, 2025.
- [28] „Scalable Polonius Support on Nightly - Rust Project Goals“, <https://rust-lang.github.io/rust-project-goals/2025h1/Polonius.html>, dostopano jan. 21, 2026.
- [29] „Rust-Lang/Rust“, <https://github.com/rust-lang/rust>, dostopano jan. 21, 2026.

- [30] „Borrow Checking in A-Mir-Formality - Rust Project Goals“, <https://rust-lang.github.io/rust-project-goals/2025h2/a-mir-formality.html>, dostopano jan. 19, 2026.
- [31] „Rust-Lang/a-Mir-Formality“, <https://github.com/rust-lang/a-mir-formality>, dostopano jan. 17, 2026.
- [32] „Minirust/Minirust“, <https://github.com/minirust/minirust>, dostopano jan. 21, 2026.
- [33] „The MIR (Mid-level IR) - Rust Compiler Development Guide“, <https://rustc-dev-guide.rust-lang.org/mir/index.html>, dostopano apr. 12, 2025.
- [34] J. Yanovski, H.-H. Dang, R. Jung, in D. Dreyer, „GhostCell: Separating Permissions from Data in Rust“, *Proceedings of the ACM on Programming Languages*, let. 5, št. ICFP, str. 1–30, avg. 2021.
- [35] „StatementKind in Rustc_middle::Mir - Rust“, https://doc.rust-lang.org/beta/nightly-rustc/rustc_middle/mir/enum.StatementKind.html, dostopano mar. 7, 2026.
- [36] „Tracking Moves and Initialization - Rust Compiler Development Guide“, https://rustc-dev-guide.rust-lang.org/borrow_check/moves_and_initialization.html#initialization-and-moves, dostopano nov. 29, 2025.
- [37] „Rvalue in Rustc_middle::Mir - Rust“, https://doc.rust-lang.org/nightly/nightly-rustc/rustc_middle/mir/enum.Rvalue.html, dostopano apr. 12, 2025.
- [38] „MIR Dataflow - Rust Compiler Development Guide“, <https://rustc-dev-guide.rust-lang.org/mir/dataflow.html>, dostopano apr. 9, 2025.
- [39] „Rust/Compiler/Rustc_borrowck/Src/Polonius at Master · Rust-Lang/Rust“, https://github.com/rust-lang/rust/tree/master/compiler/rustc_borrowck/src/polonius, dostopano apr. 12, 2025.

- [40] „TerminatorKind in Rustc_middle::Mir - Rust“, https://doc.rust-lang.org/nightly/nightly-rustc/rustc_middle/mir/enum.TerminatorKind.html, dostopano jan. 17, 2026.
- [41] „Rust-Lang/Polonius“, <https://github.com/rust-lang/polonius>, dostopano jan. 19, 2026.
- [42] „Rust-Lang/Polonius“, <https://github.com/rust-lang/polonius>, dostopano jan. 21, 2026.